

基于几何模型的烟幕干扰弹投放策略优化

摘 要

本文在烟幕干扰弹投放策略设计中,运用运动学和几何相关知识,建立了运动模型和烟幕遮挡模型,并通过变步长搜索与二分法相结合的方式计算有效遮蔽时长。针对单机单弹、单机多弹等问题建立单目标优化模型,充分挖掘遮蔽产生的隐含条件作为约束条件,对不同变量的取值进行组合寻找最优解。

针对问题一,本文首先建立**运动模型**,用于求解任意时刻各物体的位置坐标;接着建立**烟幕遮挡模型**,将目标**圆柱离散化**为上下底面圆周上的点,根据角度和相对位置关系判断目标上的点是否位于**锥形遮挡**区域内;然后,将烟幕云团 20s 的有效时间离散化,进行**变步长搜索**,根据以上两个模型判断每一时刻的遮蔽情况,较为粗略地得到有效遮蔽开始与结束时刻。最后,利用**二分法**在这两个时刻附近搜索,得到更精确的结果。最终计算得到烟幕弹对导弹 M1 的**有效遮蔽时长为 1.391641s**。

针对问题二,本文建立**单机单弹投放策略优化模型**,在 1 架无人机投放 1 枚烟幕弹干扰 1 个导弹的情况下,设计投放策略。以最大化有效遮蔽时长为优化目标,将无人机飞行方向、飞行速度、受领任务到投放烟幕弹的时间、烟幕弹投放到引爆的时间作为决策变量,建立**单目标优化模型**。从起爆时刻烟幕干扰弹、真目标圆柱和导弹的相对位置关系,产生遮蔽时云团的高度区间、导弹运动轨迹区间等方面,详细分析了产生有效遮蔽的必要条件,并将其作为约束条件缩小解空间,避免对大量非可行解计算遮蔽时长造成的算力浪费。对不同决策变量取值进行组合,采用**先粗后细**的搜索方式寻找最优解。最终投放策略为:无人机飞行方向 6.655570° ,飞行速度 129.393939m/s ,投放时间 0.816327s ,起爆延迟时间 0s ,最大有效遮蔽时长为 4.587813s 。

针对问题三,本文建立**单机多弹投放策略优化模型**,在 1 架无人机投放 3 枚烟幕弹干扰 1 个导弹的情况下设计最优投放策略。在问题二模型的基础上,新增另两枚烟幕弹的投放和起爆时间这 4 个决策变量,并增加无人机投弹时间间隔约束。考虑遮蔽时间可能不连续,修改有效遮蔽时长的求解。充分利用问题二结论,在第一枚烟幕弹最优解附近搜索可得**最大有效遮蔽时长为 6.403128s**,投放策略见 result1.xlsx。

针对问题四,本文建立**多机单弹投放策略优化模型**,在 3 架无人机各投 1 枚烟幕弹干扰 1 枚导弹的情况下设计最优投放策略。在问题三基础上,决策变量新增另两架无人机的方向和速度,约束条件减少投弹间隔约束。根据题目中三架无人机的分布特点,利用问题二方法在三枚烟幕弹最优解附近搜索可得**最大有效遮蔽时长为 11.452573s**,投放策略见 result2.xlsx。

针对问题五,对 5 架无人机每架至多投放 3 枚烟幕弹干扰 3 个导弹的情况给出最优投放策略。本文对无人机和导弹的位置进行仿真后进行**任务分配**,再利用前几问的模型进行**分组分层优化**,最终**最大有效遮蔽时长为 20.056319s**,投放策略见 result3.xlsx。

关键词: 离散化 变步长搜索 二分法 有效遮蔽

一、问题重述

1.1 问题背景

随着无人机技术的发展和信息化战争的爆发，无人机在战场上可以凭借极低的成本为我方解决大量以往难以解决的痛点问题，效费比极高。为了防止我方军事设施等重要目标被敌方导弹打击，当下常采用无人机投放烟幕干扰弹的解决措施：在特定空域巡飞的无人机接到任务后，利用精准定点投放和时序控制起爆等技术使干扰弹在特定空域起爆，产生球状烟幕云团来遮蔽我方重要目标，从而干扰敌方导弹，打击我方预设的假目标，保护真目标。因此，无人机烟幕干扰弹投放策略的制定与优化已经成为战场上保护我方重要设施的关键因素。

1.2 问题重述

无人机烟幕干扰弹投放策略包括无人机飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点等。以假目标为原点建立空间直角坐标系，5架无人机 FY1~FY5 和 3 枚导弹 M1~M3 的初始位置已知。其中无人机在接到任务后立即调整飞行方向，以 70~140m/s 的速度等高匀速直线飞行，每架无人机的飞行方向和速度可以不同，但一经确定后不再改变。无人机投放干扰弹的时间间隔至少为 1s，烟幕弹投放后在重力作用下运动，起爆后产生的球状烟幕云团由于特殊技术以 3m/s 的速度匀速下沉，有效遮蔽的空间与时间范围为云团中心 10m 以内，起爆后 20s 之内。需要建立数学模型，在真目标被遮挡的有效时间尽可能长的情况下，确定不同无人机数、不同导弹数、不同投弹数等情况下的烟幕弹投放策略。

问题一：1 架无人机 FY1 投 1 枚烟幕弹干扰 1 枚导弹 M1 时，已知无人机的飞行速度 120 m/s、方向朝向假目标、飞行 1.5s 后投放干扰弹、3.6s 后起爆，计算干扰弹对导弹 M1 的有效干扰时长。

问题二：无人机数、导弹数、烟幕干扰弹投放数与问题一相同，需要建立数学模型，在遮蔽时间尽可能长的条件下给出无人机干扰弹的投放策略。

问题三：1 架无人机 FY1 投 3 枚烟幕弹干扰 1 枚导弹 M1 时，制定无人机投放烟幕干扰弹的最优策略，将结果填入 result1.xlsx 中。

问题四：3 架无人机 FY1、FY2、FY3 各投 1 枚烟幕弹干扰 1 枚导弹 M1 时，制定无人机投放烟幕干扰弹的最优策略，将结果填入 result2.xlsx 中。

问题五：5 架无人机 FY1~FY5 每架飞机最多投 3 枚烟幕弹干扰 3 枚导弹 M1~M3 时，制定无人机投放烟幕干扰弹的策略，将结果填入 result3.xlsx 中。

二、问题分析

2.1 问题一的分析

对于问题一,首先根据题目中给出的无人机、导弹、烟幕干扰弹、烟幕云团的运动方式和物理运动学知识，可以建立运动模型，得到每个对象在任意时刻的位置坐标，为分析有效遮蔽情况奠定基础。接着，根据导弹和云团的相对位置关系和几何投影规律建立模型，将目标圆柱是否位于锥形遮挡区域内作为判断有效遮蔽的条件。然后，将圆柱离散化，用圆柱上下底面上的点来代表整个圆柱。同时将时间离散化，在时间区间内变步长搜索找到粗略的有效遮蔽开始和结束时刻，缩小搜索区

间、减小计算量；最后，在两个时刻的附近区间利用二分法搜索，精确得到有效遮蔽开始与结束的时刻，从而精确求出烟幕弹对导弹 M1 的有效干扰时长。

2.2 问题二的分析

对于问题二中一架无人机投放一枚烟幕弹干扰一个导弹的投放策略优化模型，首先根据题目条件可以确定决策变量为无人机 FY1 的飞行方向、飞行速度、从受领任务到投放烟幕弹的时间、烟幕弹投放到引爆的时间；确定目标函数为有效遮蔽时长；再根据题目中直接给出的无人机速度范围等条件和题目隐含条件确定本文模型的约束条件。进而可将问题转化为组合优化问题，将四个决策变量进行组合，计算每一时刻的有效遮蔽时长，利用约束条件和简单计算等方法得到各决策变量的大致区间，再进行精细搜索，保证结果精度。需要指出的是，为了尽可能减小模型的计算量，应尽可能充分利用约束条件和题目信息缩小需要搜索的解空间，在保证结果准确度的情况下减小计算量，从而提高模型效率。

2.3 问题三的分析

对于问题三中一架无人机投放三枚烟幕弹干扰一个导弹的投放策略优化模型，决策变量在问题二的基础上新增另外两枚烟幕弹的投放时间变量和烟幕弹从投放到起爆的时间变量；目标函数与问题二相同，仍为有效遮蔽时长；约束条件需要在问题二的基础上考虑无人机投弹的时间间隔至多 1s 的约束，考虑另外两枚烟幕弹的相关约束，进而确定问题三模型的约束条件。若要保证三枚烟幕弹的有效遮蔽时间最长，则第一枚烟幕弹的有效遮蔽时间必然为较优解，因此可以充分利用问题二的结论，在第一枚烟幕弹的最优解附近搜索，减小搜索的解空间，从而搜索得到三枚烟幕弹的最优投放策略，计算得到最长有效遮蔽时间。

2.4 问题四的分析

对于问题四中三架无人机各投放一枚烟幕弹干扰一个导弹的投放策略优化模型，决策变量在问题二的基础上需要新增另外两架无人机的方向和速度变量；目标函数与问题三相同，仍为有效遮蔽时长；约束条件在问题三的基础上不再考虑无人机投弹的时间间隔约束，进而确定问题四模型的约束条件。为了使有效遮蔽时间最长，除了要保证三枚烟幕弹的有效遮蔽时间分别较长之外，还应保证三段有效遮蔽时间段的重合部分尽可能小。因此可以充分利用问题二的模型和方法，先分别求出三架无人机所投放的三枚烟幕弹的最大有效遮蔽时间长和对应投放策略，在最优解附近展开搜索，减小搜索解空间，求出三架无人机各自投放一枚烟幕弹情况下的最大有效遮蔽时间，进而得到最优投放策略。

2.5 问题五的分析

对于问题五中五架无人机至多投放三枚烟幕弹干扰三个导弹的情况，首先可以现根据无人机和导弹的位置关系进行仿真分析，确定无人机的第一枚烟幕弹分别需要主要干扰哪个导弹，进行分组与任务分配，降低问题的维度。接着针对每组无人机的不同特点，与前几问的情况进行对应，充分利用前几问所建立的模型，分组分层优化，从而求解最大有效遮蔽时长，得到烟幕弹投放的最优策略。

三、模型假设

- 1、假设所有运动过程均忽略空气阻力；
- 2、假设没有水平风或其他因素影响，烟幕弹起爆前水平方向匀速运动，起爆后云团中心的水平位置保持不变；
- 3、忽略导弹的大小，将导弹看成一个质点；
- 4、假设导弹的视野范围足够宽，可视范围为 360° 全方位；
- 5、忽略警戒雷达发现目标到无人机接收到警戒雷达指派任务过程的时间延迟。

四、符号说明

符号	说明	单位
M_i	导弹 i 的初始位置坐标	m
v_d	导弹的速度大小	m/s
$\vec{v}_i$	导弹 i 的速度向量	-
M_i'	导弹运动 t 时间后的位置坐标	m
FY_j	无人机编号 ($j=1,2,3,4,5$)	-
F_j	无人机 j 的初始位置坐标	m
$\vec{u}_j$	无人机 j 的飞行速度向量	-
F_j'	无人机 j 经过时间 t_1 投放烟幕干扰弹的位置坐标	m
D	干扰弹初始位置坐标	m
D'	干扰弹经过时间 t_2 后起爆时的位置坐标	m
P	烟幕云团中心的初始位置坐标	m
P'	烟幕云团中心运动 t_3 时间后的位置坐标	m
v	烟幕云团的下降速度	m/s
R	烟幕云团的有效遮蔽半径	m

五、问题一模型的建立与求解

5.1 问题一模型的建立

5.1.1 运动模型

以假目标为原点建立空间直角坐标系如图 1 所示：

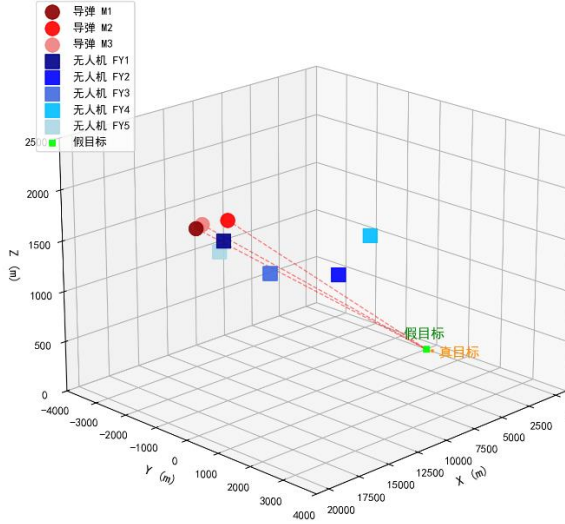


图 1 空间直角坐标系

接下来分别建立导弹、无人机、烟幕干扰弹和烟幕云团的运动模型：

(1) 导弹的运动

由题意，三枚导弹 $M_1 \sim M_3$ 的初始位置已知，设导弹 M_i 的初始坐标为 (X_i, Y_i, Z_i) ，其中 $i=1,2,3$ 。导弹的速度方向直指原点，速度大小为 $v_d = 300m/s$ ，因此导弹的速度可以用向量表示为：

$$\vec{v}_i = v_d \frac{\overrightarrow{M_i O}}{|\overrightarrow{M_i O}|} = v_d \frac{(-X_i, -Y_i, -Z_i)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} \quad (1)$$

则经过时间 t 后导弹的坐标为 $M'_i(X'_i, Y'_i, Z'_i)$ ，又因为 $\overrightarrow{M_i M'_i} = \vec{v}_i \cdot t$ ，所以经过时间 t 后导弹的坐标可以表示为：

$$M'_i \left(X_i - \frac{v_i \cdot t}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, Y_i - \frac{v_i \cdot t}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} Y_i, Z_i - \frac{v_i \cdot t}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} Z_i \right) \quad (2)$$

(2) 无人机的运动

设无人机 FY_j 的初始位置坐标为 $F_j(x_j, y_j, z_j)$ ， $j=1,2,3,4,5$ 。由于无人机等高匀速直线飞行，且飞行方向朝向假目标所在的原点，因此无人机的速度向量可以表示为：

$$\vec{u}_j = u_j \frac{(-x_j, -y_j, 0)}{\sqrt{x_j^2 + y_j^2}} \quad (3)$$

则经过时间 t 后无人机的坐标为：

$$F'_j \left(x_j - \frac{u_j \cdot t}{\sqrt{x_j^2 + y_j^2}} x_j, y_j - \frac{u_j \cdot t}{\sqrt{x_j^2 + y_j^2}} y_j, z_j \right) \quad (4)$$

(3) 烟幕干扰弹的运动

设无人机接到任务后经过时间 t_1 后投放烟幕弹，投放时刻烟幕弹坐标 D 与该时刻无人机坐标 F_j' 相同。投放后烟幕弹竖直方向在重力的作用下做自由落体运动，水平方向由于惯性保持无人机的速度做匀速直线运动。再经过 t_2 时间后在 D' 点引爆烟幕干扰弹，此时烟幕弹的坐标为：

$$D' \left(x_j - \frac{u_j \cdot (t_1 + t_2)}{\sqrt{x_j^2 + y_j^2}} x_j, y_j - \frac{u_j \cdot (t_1 + t_2)}{\sqrt{x_j^2 + y_j^2}} y_j, z_j - \frac{1}{2} g t_2^2 \right) \quad (5)$$

其中 g 为重力加速度，取 $9.8m/s^2$ 。

(4) 烟幕云团的运动

烟幕干扰弹起爆后产生球状烟幕云团，起爆时刻云团中心坐标 P 与烟幕弹起爆坐标 D' 相同。云团运动时水平位置保持不变，竖直方向以 $v = 3m/s$ 的速度匀速下沉。引爆后经过时间 t_3 ，云团中心的坐标 P' 可以表示为：

$$P' \left(x_j - \frac{u_j \cdot (t_1 + t_2)}{\sqrt{x_j^2 + y_j^2}} x_j, y_j - \frac{u_j \cdot (t_1 + t_2)}{\sqrt{x_j^2 + y_j^2}} y_j, z_j - \frac{1}{2} g t_2^2 - v t_3 \right) \quad (6)$$

综上，运动模型可以概括如图2所示：

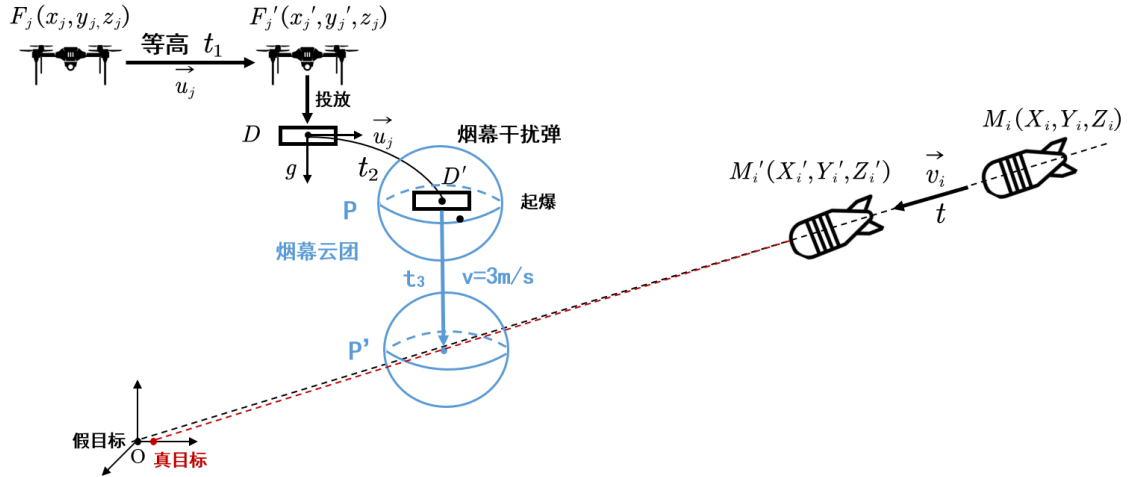


图2 运动模型

5.1.2 烟幕遮挡模型

首先对于有效遮挡的定义，本文规定整个真目标圆柱整体完全被烟幕云团遮挡住时才判定为有效遮挡。

本文将真目标圆柱体进行离散化，对圆柱上下底面进行均分，取圆柱上下底面圆周上的点来代表整个圆柱，均分方式如图 3 所示：

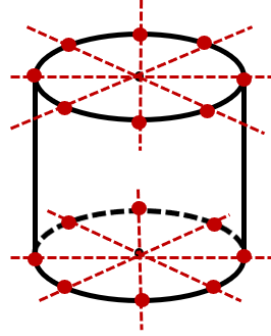


图 3 圆柱均分取点示意图

考虑到圆柱体的几何特征，对于每一条母线上的点，如果两个端点，即位于圆柱上下底面上的点同时位于遮蔽区域内，则母线中间的所有点也必然位于遮蔽区域内。因此本文只取圆柱母线两端点进行圆柱离散化，在圆周上均匀取点代表整个圆柱。该方法可以在保证结果精确度的基础上尽可能减少模型计算量，提高模型可用性。

接下来推导目标点被有效遮蔽的条件：

根据运动模型，问题一中导弹 M_1 的坐标为 $M_1(X_1, Y_1, Z_1)$ ，设云团中心坐标为 $P(a, b, c)$ ，云团半径为 R ，则由题意可知云团有效遮挡半径为 $R = 10m$ 。接下来对导弹和云团的位置进行讨论：

(1) 当导弹在云团内部时，烟幕云团必然可以有效遮蔽真目标，又因为云团为球体，由球体的解析表达式可知，此时满足条件：

$$(X - a)^2 + (Y - b)^2 + (Z - c)^2 \leq R^2 \quad (7)$$

(2) 当导弹在云团外部时，此时将云团视为一个半径 $R = 10m$ 的球状遮蔽物，则云团相对导弹产生一个锥形的遮蔽区域^[1]，如图 4 所示：

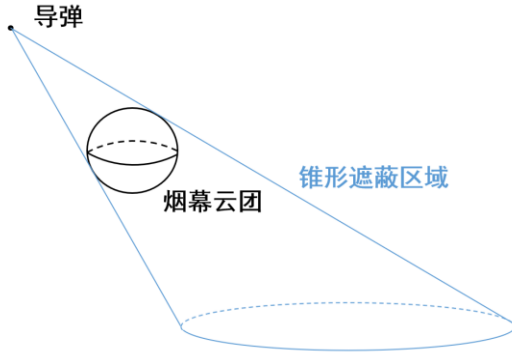


图 4 锥形遮蔽区域

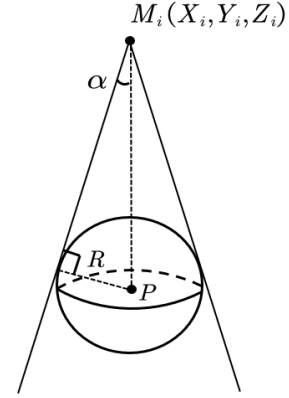


图 5 圆锥遮蔽区域几何关系

设遮蔽区域圆锥的母线与轴线的夹角为 α $\left(0 < \alpha < \frac{\pi}{2}\right)$ ，如图 5 所示。

因此，任意一点 $Q(x, y, z)$ 位于遮蔽区域内需要满足两个条件：一是向量 $\overrightarrow{MQ}$ 与的向量 $\overrightarrow{MP}$ 的夹角 β 小于 α ，即 $\cos \beta > 0$ 且 $\cos \alpha < \cos \beta$ ；二是云团要位于导弹和目标点 Q 点之间，即有 $MQ > MP \cdot \cos \alpha$ 。

由图 5 可得：

$$\sin \alpha = \frac{R}{MP} = \frac{R}{\sqrt{(X-a)^2 + (Y-b)^2 + (Z-c)^2}} \quad (8)$$

$$\cos \alpha = \sqrt{1 - \sin^2 \alpha} \quad (9)$$

因此要判断任意一点 $Q(x, y, z)$ 是否位于遮蔽区域内，只需再计算向量 $\overrightarrow{MP}$ 与向量 $\overrightarrow{MQ}$ 的夹角 β 的余弦值，即：

$$\cos \beta = \frac{\overrightarrow{MP} \cdot \overrightarrow{MQ}}{|\overrightarrow{MP}| \cdot |\overrightarrow{MQ}|} \quad (10)$$

综上，烟幕遮挡模型可以表示为：

$$\left\{ \begin{array}{l} \text{导弹在云团内} \rightarrow \text{必然为有效遮蔽} \\ \text{导弹在云团外, 任意点 } Q \text{ 为有效遮蔽} \rightarrow \left\{ \begin{array}{l} \cos \beta > 0 \text{ 且 } \cos \alpha < \cos \beta \\ MQ > MP \cdot \cos \alpha \end{array} \right. \end{array} \right. \quad (11)$$

5.2 问题一模型的求解

通过以上两种模型刻画出每个对象的运动过程、确定目标被有效遮蔽的判断标准后,本文利用 python 编程对导弹、无人机、烟幕干扰弹、烟幕云团的运动过程进行求解,从而得到烟幕弹的有效遮蔽时长。求解步骤如下:

算法:

输入: 无人机 FY_1 初始位置坐标、导弹 M_1 初始位置坐标、无人机的速度向量 $\vec{u}_j$ 、导弹的速度向量 $\vec{v}_i$ 、烟幕云团的下落速度 v 、重力加速度 g

输出: 烟幕弹对导弹 M_1 的有效遮蔽时长

Step1: 遍历有效遮蔽时间区间

将时间离散化,以一定步长 Δt 遍历时间,遍历区间为开始引爆烟幕弹到引爆后 20s,即烟幕云团能产生有效遮蔽的时间。

Step2: 计算位置坐标

根据运动模型计算每一时刻云团中心的坐标和导弹的坐标。

Step3: 判断导弹与云团的位置关系

根据烟幕遮挡模型比较导弹到云团中心的距离与云团半径的大小,判断导弹在云团的内部还是外部,若在云团内部,则认为该时刻产生有效遮挡,跳过 Step4 直接跳转到 Step1 进行下一时刻的计算;若导弹在云团外部,则进行 Step4。

Step4: 判断该时刻是否为有效遮挡

将真目标圆柱体的上下底面进行均分,取出若干个点代表整个圆柱。遍历均分后上下两个底面圆周上各点,以这些点作为烟幕遮挡模型中的 Q 点,根据有效遮挡条件判断是否位于遮挡区域。若遍历的各点均在遮挡区域内,则可认为该时刻烟幕云团产生有效遮挡。

Step5: 变步长搜索有效遮挡开始与结束时刻

若某一时刻产生有效遮挡,且前一时刻无有效遮挡,记录该时刻为 t' ;若某一时刻产生有效遮挡,且后一时刻不再产生有效遮挡,记录该时刻为 t'' 。此时 $t'' - t'$ 近似为有效遮蔽时长,其精度与搜索步长有关。若 0~20s 内均无有效遮挡时刻,缩小步长 Δt 进一步搜索。设 ε 为最大误差, Δt 缩小到 $\Delta t < \varepsilon$ 时,若仍无有效遮挡时刻,则认为有效遮蔽时长为 0。

Step6: 二分法精确计算有效遮挡时长

取区间 $[t' - \Delta t, t']$ 和 $[t'', t'' + \Delta t]$,分别用二分法进一步查找开始产生有效遮蔽的时刻 t' 和有效遮蔽结束的时刻 t'' ,当区间长度小于等于 ε 时,停止查找,此时有效遮蔽时长为 $t'' - t'$ 。

代入模型与算法，本文在真目标圆柱体上均匀取出 100 条母线，对应上下底面圆周上的 200 个点，时间遍历步长为 0.0001s，在问题一给出的烟幕干扰弹投放策略下，经过编程计算可得，烟幕弹对 M1 的有效遮蔽时长为 **1.391641s**。其中，在烟幕云团有效的 20s 时间内，遮蔽开始的时间为第 8.056445s，遮蔽结束的时间为第 9.448086s。结果展示如表 1 所示：

表 1 问题一计算结果

有效遮蔽时长	遮蔽开始时间	遮蔽结束时间
1.391641s	8.056445s	9.448086s

分析结果可知，有效遮蔽时长只占烟幕云团有效 20s 的 6.95%，遮蔽时间的利用率较低，因此需要建立优化模型，对烟幕弹的有效遮蔽时长进行优化。

六、问题二模型的建立与求解

6.1 单机单弹投放策略优化模型的建立

根据题目要求，需要在一架无人机投放一枚烟幕弹干扰一个导弹的情况下，设计无人机投弹的投放策略，使得烟幕弹的有效遮蔽时间尽可能长。因此建立以下单机单弹投放策略优化模型：

6.1.1 决策变量

针对本题目，决策变量共有四个，包括无人机 FY1 的飞行方向、无人机 FY1 的飞行速度、无人机受领任务到投放烟幕弹的时间、烟幕弹投放到引爆的时间。

- 无人机 FY1 的飞行方向： θ ，规定以 x 轴为正向，逆时针方向为正；
- 无人机 FY1 的飞行速度： u ；
- 无人机受领任务到投放烟幕弹的时间： t_1 ；
- 烟幕弹投放到引爆的时间： t_2 ；

由问题一中的运动模型可知，上述四个变量一旦确定，则烟幕干扰弹投放点、烟幕干扰弹起爆点的位置也随之确定，烟幕干扰弹的投放策略即可确定。

6.1.2 目标函数

模型的优化目标是令有效遮蔽时长尽可能大。有效遮蔽时长的计算方法与问题一相同，具体计算方法和步骤可见前文“问题一模型的建立与求解”。

6.1.3 约束条件

对于约束条件的规定，除了题目现有要求外，对于投放策略的一组解 (θ, u, t_1, t_2) ，

要使其存在产生有效遮蔽的可能，可以给出一些隐含的约束条件，从而极大地减小需要搜索的解空间范围，减小计算量，提高模型计算效率。下面具体分析约束条件：

(1) 无人机飞行方向角度在 0 到 2π 之间：

$$\theta \in [0, 2\pi) \quad (12)$$

(2) 无人机飞行速度在 70m/s 到 140m/s 范围内：

$$70 \leq u \leq 140 \quad (13)$$

(3) 无人机受领任务到投放烟幕弹的时间和烟幕弹投放到引爆的时间非负：

$$t_1 \geq 0, t_2 \geq 0 \quad (14)$$

(4) 起爆时刻烟幕干扰弹与真目标、导弹的相对位置关系的约束：

由问题一运动模型，无人机匀速等高飞行，则无人机的速度向量可以表示为：

$$\vec{u} = (u \cdot \cos\theta, u \cdot \sin\theta, 0) \quad (15)$$

则起爆时刻烟目干扰弹的位置坐标，即烟幕云团中心的初始位置坐标为：

$$P\left(x + (t_1 + t_2) \cdot u \cdot \cos\theta, y + (t_1 + t_2) \cdot u \cdot \sin\theta, \frac{1}{2}gt_2^2\right) \quad (16)$$

其中 x 、 y 为无人机初始位置的 x 、 y 坐标。

此时导弹 i 的位置坐标为：

$$M_i'\left(X_i - \frac{v_d \cdot (t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, Y_i - \frac{v_d \cdot (t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} Y_i, Z_i - \frac{v_d \cdot (t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} Z_i\right) \quad (17)$$

其中 X_i 、 Y_i 、 Z_i 为导弹 i 的初始位置坐标， v_d 为导弹飞行速度 300m/s 。

将起爆点、导弹位置、导弹运动轨迹、真目标圆柱底面圆心在 xoy 平面上投影如图 6 所示：

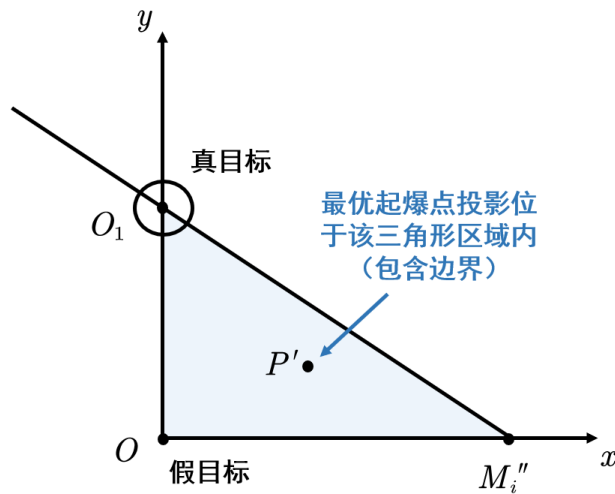


图 6 xoy 平面内投影位置关系

由图可知，真假目标投影点连线 OO_1 、导弹轨迹投影直线 OM_i'' 、起爆时刻导弹投影与真目标地面圆心连线 O_1M_i'' ，这三条直线构成的三角形将最优起爆点约束在该三角形区域内。由图易知，三角形外的起爆点必然不能产生有效遮蔽，即使在三条边附近的某些点有可能产生遮蔽，但其遮蔽时长一定不是最大的，因此可以认为**最优起爆点投影必然落在该三角形区域内（包含边界）**，即起爆点的横坐标 $x_{P'}$ 位于原点和导弹横坐标 $x_{M_i''}$ 之间，且起爆点的纵坐标夹在直线 OM_i'' 和直线 O_1M_i'' 之间。

进一步，设直线 OM_i'' 的方程为 $y = k_1x$ ，直线 O_1M_i'' 的方程为 $y = k_2x + 200$ 。将导弹的初始位置坐标代入即可得到 k_1 ，而 k_2 可以用决策变量 t_1 ， t_2 来表示，则该约束条件可以表示为：

$$\begin{cases} 0 \leq x_{P'} \leq x_{M_i''} \\ k_1 x_{P'} \leq y_{P'} \leq k_2 x_{P'} + 200 \end{cases} \quad (18)$$

（5）起爆点高度的约束：

如图 7 所示，过导弹运动轨迹直线可以作两个与目标圆柱体的切面，两个切面的方程固定且可以根据题目已知条件直接求出。

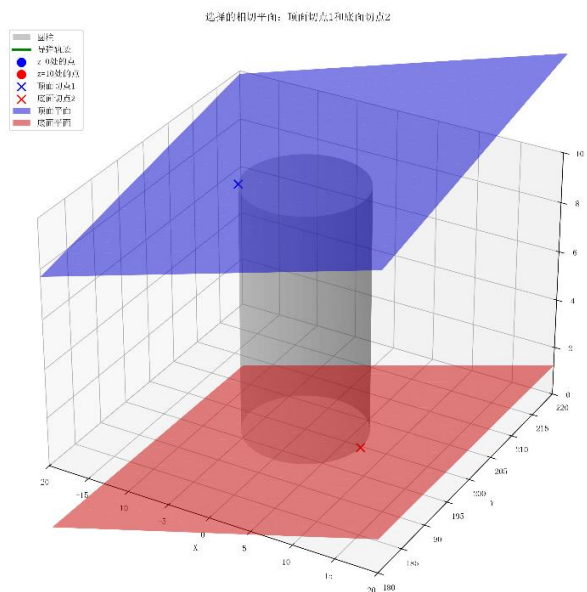


图 7 导弹轨迹直线与目标圆柱上下底面的切面

由几何知识可知，导弹轨迹直线与目标圆柱的两个切平面实际是直线与目标圆柱上下底面圆的切平面。定义过导弹轨迹直线与目标圆柱上底面圆相切的平面为上切面，与下底面圆相切的平面为下切面。

如图 8 所示，由几何关系可知：如果一个平面与圆面相切，则该平面与圆面所在

平面的交线是圆的切线。因此，由于下底面圆与所在平面与导弹轨迹直线的交点为原点，则原点在下切面与下底面的交线上，由前面的结论可知，该交线必与下底面圆相切，即在 xoy 平面上过原点作下底面圆的切线，则该切线落在下切面上。

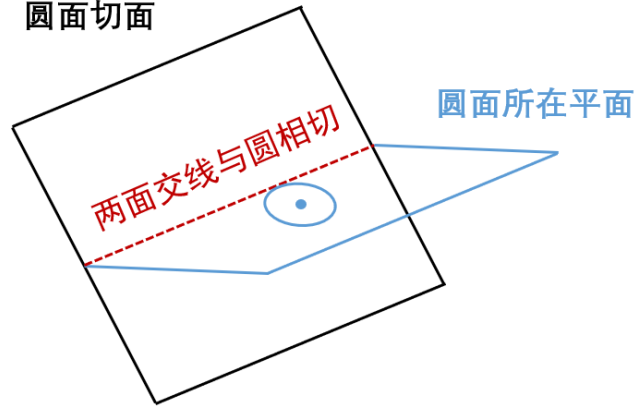


图 8 几何结论

又因为两条相交直线确定一个平面，则根据切线和导弹轨迹直线方程即可确定下切面的方程。同理，令 $z=10$ 求得上底面与导弹轨迹直线的交点，进而求得过该点与上底面圆的切线，由切线和导弹轨迹确定上切面方程。

确定上下切面方程的步骤如下：

首先，分别过上下底面与导弹轨迹直线的交点作上下底面圆的切线，求出两条切线的方程：

接着，设切线斜率为 k ，则三维坐标系中切线的方向向量为：

$$\vec{d} = \left(\frac{1}{\sqrt{1+k^2}}, \frac{k}{\sqrt{1+k^2}}, 0 \right) \quad (19)$$

然后，可得到切平面的法向量为 $\vec{n} = \vec{d} \times \overrightarrow{OM}$ ，带入导弹与向量坐标整理可得：

$$\vec{n} = \left(\frac{kZ}{\sqrt{1+k^2}}, -\frac{Z}{\sqrt{1+k^2}}, \frac{Y-kX}{\sqrt{1+k^2}} \right) \quad (20)$$

最后，因为切平面过原点，则切平面方程为：

$$\vec{n} \cdot (x, y, z) = 0 \quad (21)$$

此外，由于过一点作圆的切线有两条，因此上述过程所求的上切面和下切面均有两个，需进一步检验与筛选：将 $x=0, y=200$ 代入切面方程，求出 z ，由几何关系可知，若 $0 < z < 10$ ，则舍去该切面方程。最终可以得到上下切面的方程。

接下来分析烟幕云团与切面的位置关系并计算相切时云团中心的高度：

设云团下方与下切面相切时云团中心的高度为 H_1 ，云团上方与上切面相切时中心

高度为 H_2 ，则只有当云团中心的高度在区间 $[H_2, H_1]$ 内时，烟幕云团才有可能产生有效遮蔽。

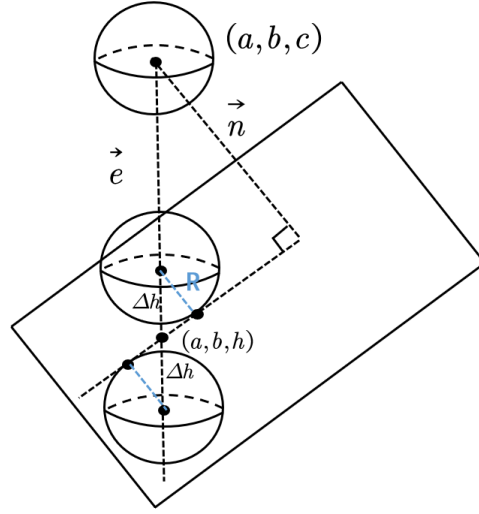


图9 烟幕云团与切平面相切

如图9所示，烟幕云团中心坐标为 (a, b, c) ，设云团中心的运动轨迹与切平面的交点为 (a, b, h) ，云团中心所在直线与向量 $\vec{e} = (0, 0, 1)$ 共线，切平面的法向量为 $\vec{n}$ ，则由几何关系可得：

$$\frac{R}{\Delta h} = \frac{|\vec{e} \cdot \vec{n}|}{|\vec{n}|} \quad (22)$$

由上式可解得 Δh 。再由对称关系可得，相切时云团中心的坐标为 $(a, b, h \pm \Delta h)$ ，计算与上切面相切时取减号，与下切面相切时取加号，再根据起爆点的 x 、 y 坐标可以整理得到 H_1 和 H_2 的表达式。

同时，可以求出云团中心在烟幕弹起爆后的有效时间内（0~20s）的高度区间为：

$$\left[z - \frac{1}{2}gt_2^2 - 20v, z - \frac{1}{2}gt_2^2 \right] \quad (23)$$

其中 v 为烟幕云团下降的速度3m/s， t_2 为烟幕弹投放到引爆的时间， z 为无人机初始位置高度。因此，只有该区间与 $[H_2, H_1]$ 区间有交集时，才有可能产生有效遮蔽，如图10所示，则该约束条件可以表示为：

$$\begin{cases} z - \frac{1}{2}gt_2^2 > H_2 \\ z - \frac{1}{2}gt_2^2 - 20v < H_1 \end{cases} \quad (24)$$

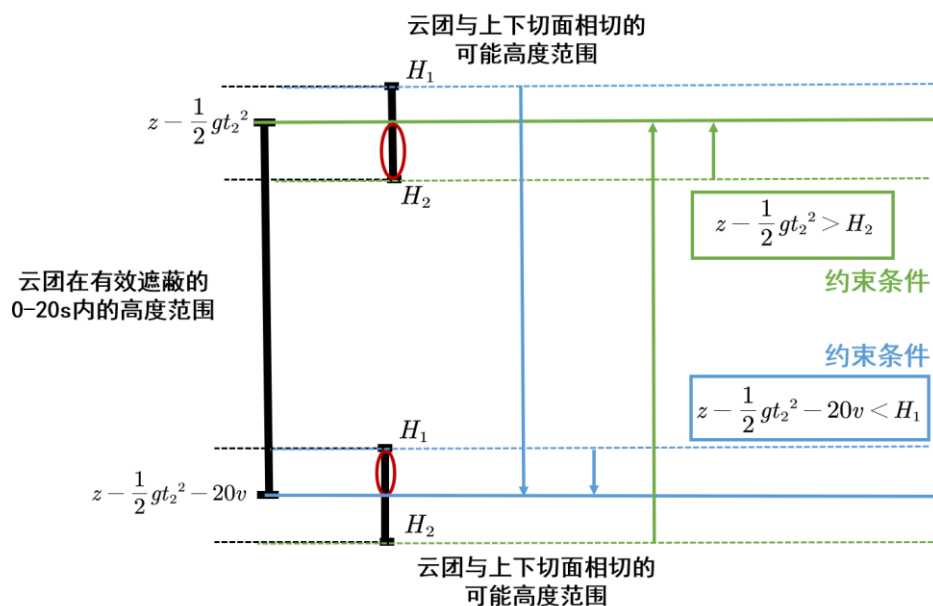


图 10 高度区间约束条件

(6) 导弹位置的约束:

由运动模型可知，烟幕云团水平位置不变，竖直方向做匀速直线运动，因此云团的运动轨迹可形成一个圆柱面，可以做出真目标圆柱体与云台轨迹圆柱的两个切平面。图 11 展示了俯视效果图:

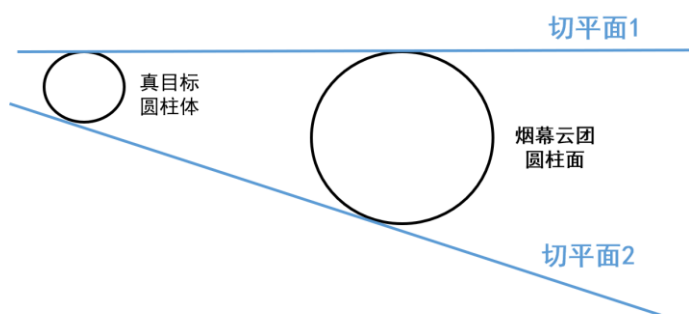


图 11 两圆柱切面示意图

同时，设导弹运动轨迹为直线 l ，直线 l 与两个圆柱的切平面相交，交点分别设为 A、B（假设导弹先到达 A 点），如图 12 所示。

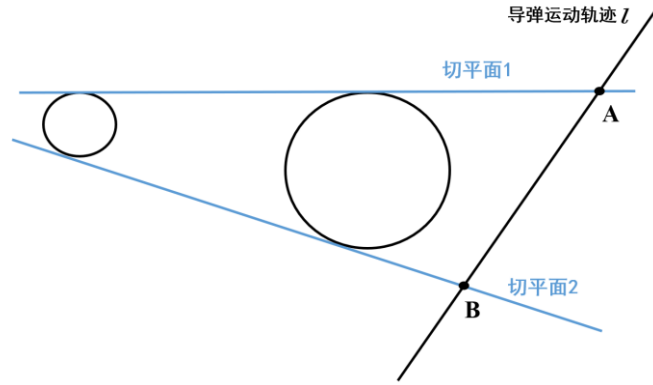


图 12 导弹轨迹直线与两圆柱切面的交点

则由几何关系可知，只有当导弹位于线段 AB 上时，烟幕云团才有可能形成有效遮蔽。进一步需要求出 A、B 点的坐标：

由几何关系可知，目标圆柱与云团轨迹柱面的切平面垂直于 xoy 平面，因此求三位坐标系中的切平面方程可简化为求 xoy 二维平面中的切线方程。如图 13 所示：

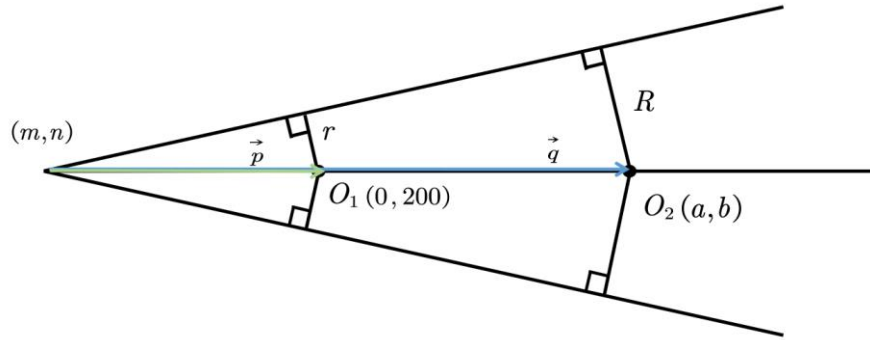


图 13 简化为 xoy 平面内求切线

其中 O_1 和 O_2 分别代表目标圆柱体底面圆心和云团中心投影，由题意可知位置坐标 $O_1(0, 200)$ ， $O_2(a, b)$ 。设两条切线的交点坐标为 (m, n) ，设向量 $\vec{p}$ 为两切线交点指向目标圆柱底面圆心的向量， $\vec{q}$ 为两切线交点指向云团中心投影的向量，则可表示为：

$$\vec{p} = (-m, 200 - n) \quad (25)$$

$$\vec{q} = (a - m, b - n) \quad (26)$$

根据三角形相似的比例关系可得：

$$\vec{q} = \frac{R}{r} \vec{p} \quad (27)$$

带入向量 $\vec{p}$ 和 $\vec{q}$ 的坐标可得：

$$\begin{cases} a - m = \frac{R}{r}(-m) \\ b - n = \frac{R}{r}(200 - n) \end{cases} \quad (28)$$

解方程组可得：

$$\begin{cases} m = \frac{ra}{r - R} \\ n = \frac{rb - 200R}{r - R} \end{cases} \quad (29)$$

由解析几何知识可知，过点 (m, n) 的直线方程为 $y - n = k(x - m)$ ，因为 $O_1(0, 200)$ 到该直线的距离为 r ，因此可得：

$$\frac{|k(-m) - 200 + n|}{\sqrt{1 + k^2}} = r \quad (30)$$

解方程可以得到两切线的斜率 k_1 、 k_2 ，则两个切平面的方程分别为：

$$y - n = k_1(x - m) \quad (31)$$

$$y - n = k_2(x - m) \quad (32)$$

通过上述方式可以由起爆点的 x 、 y 坐标求得切平面方程，联立切平面方程和导弹轨迹方程可以解出 A 、 B 点坐标。

因为烟幕干扰弹起爆后 $20s$ 内导弹的横坐标区间为：

$$\left[X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i \right] \quad (33)$$

位置关系如图 14 所示：

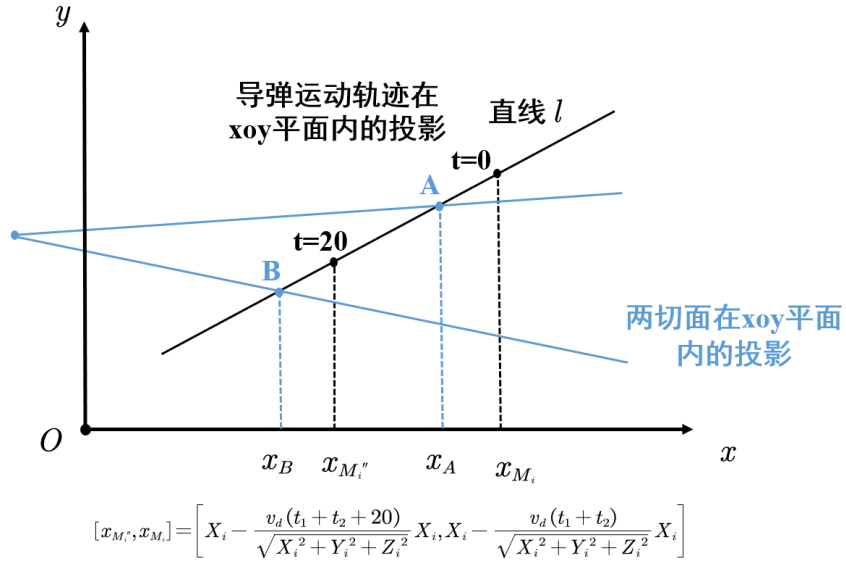


图 14 高度区间交集

因此该约束条件可以表示为：

$$\begin{cases} X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i < x_A \\ X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i > x_B \end{cases} \quad (34)$$

(7) 时间约束：

若一组解满足约束（5）和（6），则还需满足云团中心经过高度 $[H_2, H_1]$ 的时间区间与导弹经过 AB 的时间区间有交集，才有可能产生有效遮蔽。

如图 15 所示，烟幕云团中心的高度区间 $\left[z - \frac{1}{2}gt_2^2 - 20v, z - \frac{1}{2}gt_2^2 \right]$ 与 $[H_2, H_1]$ 的交集可以表示为：

$$\left[\max\left(z - \frac{1}{2}gt_2^2 - 20v, H_2\right), \min\left(z - \frac{1}{2}gt_2^2, H_1\right) \right] \quad (35)$$

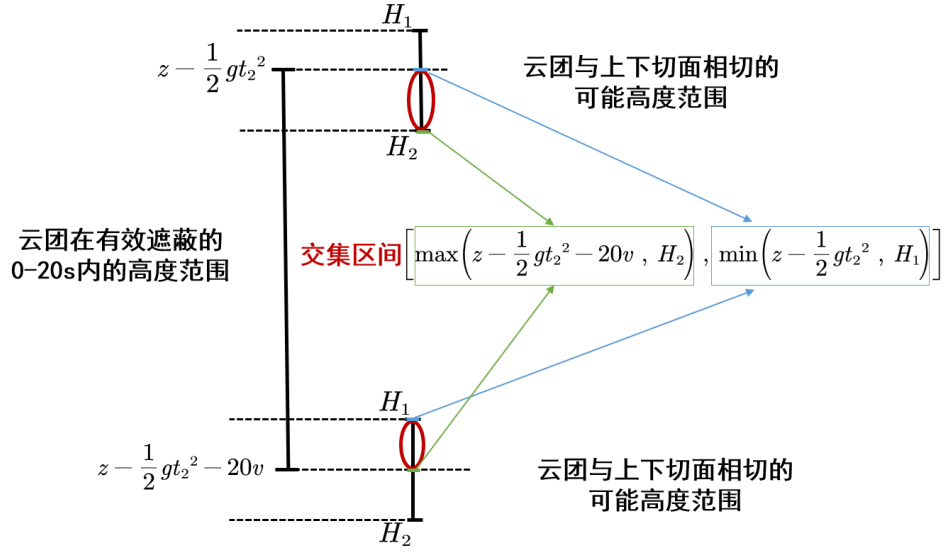


图 15 高度区间交集

由于云团中心竖直方向做 $v = 3m/s$ 的匀速直线运动，则该高度区间交集对应的时间区间为：

$$\left[\frac{\min\left(z - \frac{1}{2}gt_2^2, H_1\right) - \left(z - \frac{1}{2}gt_2^2\right)}{v}, \frac{\max\left(z - \frac{1}{2}gt_2^2 - 20v, H_2\right) - \left(z - \frac{1}{2}gt_2^2\right)}{v} \right] \quad (36)$$

又因为导弹在烟幕云团有效遮挡的 20s 内的横坐标运动变化区间 (33) 和导弹轨迹直线与两圆柱切面的两个交点横坐标区间 $[x_B, x_A]$ 的交集为：

$$\left[\max\left(X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_B\right), \min\left(X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_A\right) \right] \quad (37)$$

由于导弹的运动速度 $v_d = 300m/s$ ，则该交集对应的时间区间为：

$$\left[\frac{\min\left(X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_A\right) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2), \frac{\max\left(X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_B\right) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \right] \quad (38)$$

由于时间区间 (36) 和 (38) 需要存在交集，才可能对真目标进行有效遮蔽，因此该约束条件可表示为：

$$\begin{cases} \frac{\max\left(z - \frac{1}{2}gt_2^2 - 20v, H_2\right) - \left(z - \frac{1}{2}gt_2^2\right)}{v} > \frac{\min\left(X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_A\right) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \\ \frac{\min\left(z - \frac{1}{2}gt_2^2, H_1\right) - \left(z - \frac{1}{2}gt_2^2\right)}{v} < \frac{\max\left(X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_B\right) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \end{cases} \quad (39)$$

(8) 其他约束:

- 烟幕干扰弹在落地之前要引爆:

$$\frac{1}{2}gt_2^2 < z \quad (40)$$

- 烟幕干扰弹在导弹击中假目标之前要引爆:

$$t_1 + t_2 < \frac{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}{v_d} \quad (41)$$

综上, 单机单弹投放策略优化模型可总结如下, 其中有效遮蔽时长为 T , 计算方法同问题一:

$$\begin{aligned} & \max T \\ & \left\{ \begin{array}{l} 0 \leq \theta \leq 2\pi \\ 70 \leq u \leq 140 \\ t_1 \geq 0, t_2 \geq 0 \\ 0 \leq x_{P'} \leq x_{M_i} \\ k_1 x_{P'} \leq y_{P'} \leq k_2 x_{P'} + 200 \\ z - \frac{1}{2}gt_2^2 > H_2 \\ z - \frac{1}{2}gt_2^2 - 20v < H_1 \\ X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i < x_A \\ X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i > x_B \\ \frac{\max\left(z - \frac{1}{2}gt_2^2 - 20v, H_2\right) - \left(z - \frac{1}{2}gt_2^2\right)}{v} \dots \\ > \frac{\min\left(X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_A\right) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \\ \frac{\min\left(z - \frac{1}{2}gt_2^2, H_1\right) - \left(z - \frac{1}{2}gt_2^2\right)}{v} \dots \\ < \frac{\max\left(X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_B\right) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \\ \frac{1}{2}gt_2^2 < z \\ t_1 + t_2 < \frac{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}{v_d} \end{array} \right. \end{aligned}$$

6.2 问题二模型的求解

由于本题目对烟幕弹投放策略的优化涉及无人机飞行角度、无人机飞行速度、无人机从任务受领到投放的时间、烟幕弹起爆时间这四个决策变量的优化，因此属于组合优化问题。由于有效遮蔽时长随四个决策变量的变化是缓变的，可以先确定每个决策变量的取值集合，在对不同的决策变量进行组合，然后遍历不同组合，计算有效遮蔽时长，寻找到有效遮蔽时长最长的即为最优解。^[2]本文算法分为以下两个步骤：

● **根据约束条件压缩决策变量取值范围**

计算过程中，对于每一组决策变量，先判断该组解是否满足约束条件，若满足约束条件，再利用问题一的模型求解有效遮蔽时长。在采用问题一的时间离散化遍历中，由于约束条件（7）已经确定了可能产生有效遮蔽的时间范围，因此可以用这一时间区间来代替问题一模型中的 $[0,20]$ 区间，减小搜索区间的长度，加快有效遮蔽时长的求解效率。根据约束条件和简单计算模拟可以先确定各个决策变量的取值范围：

表 2 各个决策变量的取值范围

决策变量	取值范围	区间取点数
无人机 FY1 的飞行方向 θ	$[0, 28.64789]$	100
无人机 FY1 的飞行速度 u	$[70, 140]$	100
受领任务到投放烟幕弹的时间 t_1	$[0, 20]$	50
烟幕弹投放到引爆的时间 t_2	$[0, 20]$	50

● **分阶段由粗到细搜索最优解**

对于以上得到的大致取值范围，利用问题一的模型求出每一时刻的有效遮蔽时长，求解时本文首先采用简化模型，即将目标圆柱体视为两个点，当两个点都位于遮蔽区域内时判定为有效遮蔽，找到有效遮蔽时长较大的区间，即为各个决策变量的较优解区间，快速缩小需要搜索的区间，减小计算量。接着在这个较小的区间中进行精细搜索，此时不再简化模型，仍将圆柱均分成 200 点求解有效遮蔽时长，保证结果精度。

最终本文得到的投放策略及最大有效遮蔽时长如表 3 所示：

表 3 投放策略及最大有效遮蔽时长

飞行方向 ($^{\circ}$)	飞行速度 (m/s)	投放时间 (s)	起爆时间 (s)	最大有效遮蔽时长 (s)
6.655570	129.393939	0.816327	0	4.587813

分析结果可知，在相同情况下，单机单弹投放策略优化模型将有效遮蔽时长从问题一的 1.391641s 增大到问题二的 4.587813s，有效遮蔽时长增大约 3.3 倍，证明本文建立的单机单弹投放策略优化模型可以在尽可能减小搜索区间与计算量的同时，显著提高有效遮蔽时长，模型效率较高，对于实际应用具有重要意义。

七、问题三模型的建立与求解

7.1 单机多弹投放策略优化模型的建立

对于问题三单架无人机投放三枚烟幕弹的情况，在问题二模型的基础上建立单机多弹投放策略优化模型，设计烟幕干扰弹的投放策略，使有效遮蔽时间尽可能长。

7.1.1 决策变量

该模型的决策变量在问题二的基础上增加了另外两枚烟幕弹的投放时间变量和烟幕弹从投放到起爆的时间变量：

- 无人机 FY1 的飞行方向： θ ；
- 无人机 FY1 的飞行速度： u ；
- 从受领任务到第一、二、三枚弹的投放时间： t_{11} 、 t_{12} 、 t_{13} ；
- 三枚烟幕干扰弹从投放到引爆的时间： t_{21} 、 t_{22} 、 t_{23} ；

7.1.2 目标函数

该目标的优化目标仍为使得有效遮蔽时间尽可能长，但有效遮蔽时长的计算与问题一略有不同，在问题一的基础上做出如下改动：

- **有效遮蔽判定条件修改**

由于问题三需要投放三枚烟幕干扰弹，对于任意时刻 t ，在已经引爆的烟幕干扰弹中，只要有一枚烟幕干扰弹能形成有效遮蔽，则判定该时刻产生有效遮蔽；

- **有效遮蔽时长 T 计算方法修改**

由于有效遮蔽的时间区间可能不连续，即可能存在多个有效遮蔽的起止时刻 t' 和 t'' 。按照时间的先后顺序对每一组 t' 和 t'' 分别求有效遮蔽时长，最终有效遮蔽时长为各组的有效遮蔽时长之和。

7.1.3 约束条件

问题三优化模型的约束条件在问题二约束条件的基础上进行修改，具体分析过程见问题二“单机单弹投放策略优化模型的建立”，本节主要指出需要修改之处：

(1) 无人机飞行方向角度在 0 到 2π 之间：

$$\theta \in [0, 2\pi) \quad (42)$$

(2) 无人机飞行速度在 70m/s 到 140m/s 范围内:

$$70 \leq u \leq 140 \quad (43)$$

(3) 除了无人机从受领任务到第一、二、三枚弹的投放时间非负, 还需考虑无人机投弹时间间隔不超过 1s, 则约束条件 (3) 修改为:

$$\begin{cases} t_{11}、t_{12}、t_{13} \geq 0 \\ t_{21}、t_{22}、t_{23} \geq 0 \\ t_{12} - t_{11} \geq 1 \\ t_{13} - t_{12} \geq 1 \end{cases} \quad (44)$$

约束条件 (4) (5) (6) (7) 均与问题二相同, 下面直接列出约束条件:

(4) 起爆时刻, 烟幕干扰弹与真目标、导弹的相对位置关系的约束:

$$\begin{cases} 0 \leq x_{P'} \leq x_{M_i''} \\ k_1 x_{P'} \leq y_{P'} \leq k_2 x_{P'} + 200 \end{cases} \quad (45)$$

(5) 起爆点高度的约束:

$$\begin{cases} z - \frac{1}{2}gt_2^2 > H_2 \\ z - \frac{1}{2}gt_2^2 - 20v < H_1 \end{cases} \quad (46)$$

(6) 导弹位置的约束:

$$\begin{cases} X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i < x_A \\ X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i > x_B \end{cases} \quad (47)$$

(7) 时间约束:

$$\begin{cases} \frac{\max(z - \frac{1}{2}gt_2^2 - 20v, H_2) - (z - \frac{1}{2}gt_2^2)}{v} > \frac{\min(X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_A) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \\ \frac{\min(z - \frac{1}{2}gt_2^2, H_1) - (z - \frac{1}{2}gt_2^2)}{v} < \frac{\max(X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_B) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \end{cases} \quad (48)$$

(8) 其他约束:

- 三枚烟幕干扰弹在落地之前要引爆:

$$\frac{1}{2}gt_{21}^2 < z, \quad \frac{1}{2}gt_{22}^2 < z, \quad \frac{1}{2}gt_{23}^2 < z \quad (49)$$

- 三枚烟幕干扰弹在导弹击中假目标之前要引爆:

$$t_{11} + t_{21} < \frac{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}{v_d}, \quad t_{12} + t_{22} < \frac{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}{v_d}, \quad t_{13} + t_{23} < \frac{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}{v_d} \quad (50)$$

综上, 问题三单机多弹投放策略优化模型总结如附录 2 所示。

7.2 问题三模型的求解

由计算分析可知, 当三枚烟幕弹产生的有效遮蔽时长最大时, 第一枚烟幕弹的有效遮蔽时长必然为一个较优值, 因此可以充分利用问题二的模型算法与结果, 在问题二求解得到的最优解附近展开搜索, 极大缩减需要搜索的解空间范围。最终计算得到最大有效遮蔽时长为 **6.403128s**, 最优投放策略见 result1.xlsx。

八、问题四模型的建立与求解

8.1 多机单弹投放策略优化模型的建立

对于问题四有三架无人机各投一枚烟幕弹干扰一枚导弹的情况, 本文在问题二和问题三的基础上建立多机单弹投放策略优化模型, 给出有效遮蔽时间最长时的无人机烟幕弹投放策略。

8.1.1 决策变量

问题四优化模型在问题二的基础上增加了另外两架无人机的飞行角度、速度以及各自投放的三枚烟幕弹的投放时间和起爆延迟时间。

- 三架无人机 FY1、FY2、FY3 的飞行方向: θ_1 、 θ_2 、 θ_3
- 三架无人机 FY1、FY2、FY3 的飞行速度: u_1 、 u_2 、 u_3
- 三架无人机 FY1、FY2、FY3 从受领任务到投放烟幕弹的时间: t_{11}' 、 t_{12}' 、 t_{13}'
- F 三架无人机 Y1、FY2、FY3 从投放到烟幕弹起爆的时间: t_{21}' 、 t_{22}' 、 t_{23}'

8.1.2 目标函数

问题四优化目标仍为使有效遮蔽时间尽可能长, 有效遮蔽时长的计算与求解过程与问题三相同, 具体见问题三“单机多弹投放策略优化模型的建立”。

8.1.3 约束条件

问题四模型的约束条件与问题三大致相同，由于每架无人机只投 1 枚烟幕弹，无需考虑无人机投弹时间间隔的约束，但需要三架无人机各自投放的烟幕弹均满足问题三中（4）（5）（6）（7）的约束条件。

8.2 问题四模型的求解

如图 16 中三架无人机 FY1、FY2、FY3 的位置与导弹 M1 的位置关系可直观看出，三枚烟幕弹产生遮蔽的先后顺序为 FY1、FY2、FY3。

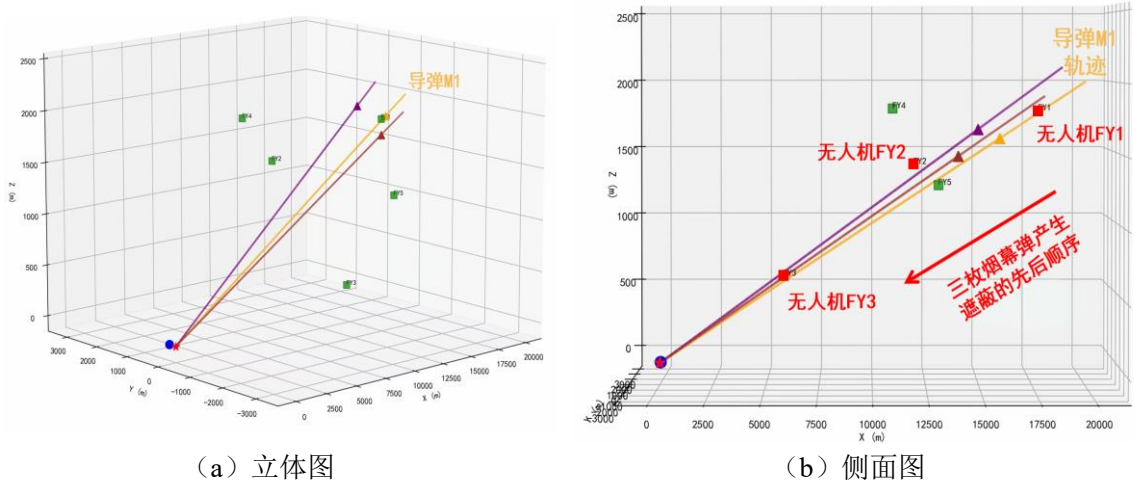


图 16 问题四中三架无人机与导弹轨迹的位置关系

为了保证最终总的有效遮蔽时间最长，在保证三枚烟幕弹各自有效遮蔽时间尽可能长的基础上，还应该保证三个有效遮蔽时间段的重叠部分尽可能小，因此可以先用问题二的模型求解出每架无人机能产生的最大有效遮蔽时长以及对应的有效遮蔽时间段，再在问题二求得的三架无人机的最优解的组合附近展开搜索，极大缩减待搜索的解空间范围，减小计算量，极大提高模型效率。三架无人机的最优解如表 4 所示：

表 4 FY1、FY2、FY3 三架无人机的各自最优投放策略与有效遮蔽时长

	FY1	FY2	FY3
飞行方向 (°)	6.655570	312.727273	125.635918
飞行速度 (m/s)	129.393939	105.353535	131.984652
投放时间 (s)	0.816327	14.285714	30.461583
起爆时间 (s)	0	3.673469	
有效遮蔽时长 (s)	4.587813	3.850156	2.796243

最终计算得到最大有效遮蔽时长为 11.452573s，最优投放策略见 result2.xlsx。

九、问题五的分析与求解

针对问题五中 FY1~FY5 五架无人机每架至多投放三枚烟幕弹干扰三枚 M1~M3 导弹的情况，本文通过仿真计算^[3]与分析，将无人机与需要干扰的导弹进行对应组合、

任务分配，再综合利用问题二、问题三、问题四中的模型与方法来求解问题五。具体分析如下：

首先，问题五的优化目标是使得三枚导弹各自的有效遮蔽时长之和最大，计算时利用问题一模型和方法对三枚导弹的有效遮蔽时长分别求解再相加即可。

接着，分析无人机干扰导弹的任务分配与组合情况：

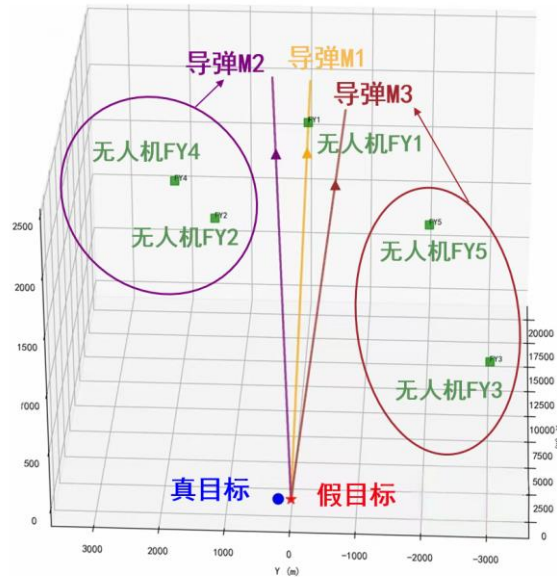


图 17 无人机与导弹位置分析

如图 17 所示，三枚导弹的位置和来袭方向相对集中，在图示角度中，导弹 M1 位于中间位置，导弹 M2 位于左侧，导弹 M3 位于右侧。而无人机 FY2 和 FY4 在三枚导弹运动轨迹整体的左侧，FY3 和 FY5 在三枚导弹运动轨迹的右侧。因此，初步确定投放策略为位于左侧的无人机 FY2 和 FY4 的第一枚烟幕弹用于干扰左侧导弹 M2，位于右侧的无人机 FY3 和 FY5 的第一枚烟幕弹用于干扰右侧导弹 M3。

由无人机 FY1、导弹 M1 的初始位置坐标和运动模型可知，无人机 FY1 与导弹 M1 的运动轨迹共面，进一步通过对运动模型的仿真模拟^[3]发现，无人机 FY1 投放的烟幕弹对导弹 M3 不可能产生遮挡，对导弹 M2 可能产生遮蔽的时长较短，接下来具体分析原因：

如图 18 所示为 $t=3.68s$ 时刻垂直于 xoy 平面视角的无人机与导弹的运动情况。由于无人机作匀速等高运动，且速度范围为 $70\sim140m/s$ ，则可以将无人机每一时刻的可能到达位置进行可视化处理，如图所示，绿色圆环为无人机以 $70m/s$ 的速度飞行时各个方向到达的位置，即最近到达位置，蓝色圆环为飞行速度 $140m/s$ 时对应的最远到达距离，两圆环所夹区域即为无人机可能到达的位置。

由前几问投放策略优化模型中的约束条件（4）可知，云团中心在 xoy 平面中的投影应位于真目标底面圆心、原点和导弹坐标投影所构成的三角形区域内部，才有可能产生较长的有效遮蔽时间。但如图 18 所示，由于导弹速度大于无人机的速度，在无人机 FY1 到达导弹 M3 运动轨迹之前 M3 就已经飞过 FY1 可能到达的位置，无人机投放烟幕弹产生的云团中心投影不可能位于该三角形之内，不满足约束条件（4），无法对导弹 M3 形成有效遮挡。

因此无论无人机 FY1 如何投放烟幕弹都无法对导弹 M3 产生有效遮挡。同理，无

人机 FY1 对导弹 M2 的遮挡时间也极短。因此，无人机 FY1 只用来干扰导弹 M1，即 FY1 的烟幕弹投放策略只针对导弹 M1，可以直接沿用问题三的结果。

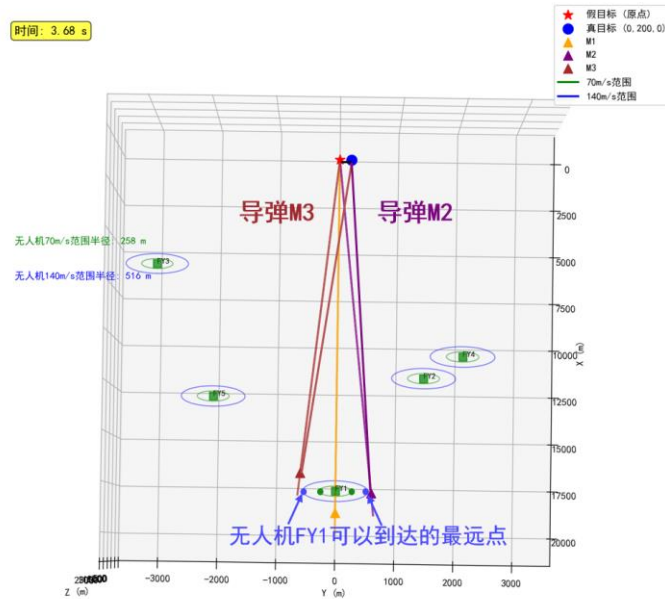


图 18 无人机运动情况分析

此外，由于无人机 FY3 和 FY5 的距离较远，高度相差较大，对二者的运动进行仿真模拟^[3]发现，单看约束条件（4）时，两架无人机投放的烟幕弹对导弹 M3 同时产生有效遮蔽的时间极短，说明二者产生有效遮蔽的时间区间重叠的概率极低，因此可以将 FY3 和 FY5 两架无人机单独考虑，分别进行优化。

而对于无人机 FY2 和 FY4，二者距离较近，在仿真模拟的过程中发现，在可能产生有效遮蔽时，两架无人机的位置范围有较多重叠，因此无人机 FY2 和 FY4 的参数需要同时考虑，同步优化。

针对以上不同的分组情况，问题五的优化步骤如下：

首先，对于无人机 FY1 只需干扰一枚导弹 M1 的情况，直接沿用问题三的投放策略即可，可直接得到有效遮蔽时长和对应的有效遮蔽时间区间；

接着，针对无人机 FY3 和 FY5 进行单独优化投放策略，属于单机三单多干扰目标的情况，模型与计算方法与问题三大致相同。不同之处为模型的目标函数需要改为烟幕云团对三枚导弹各自被遮蔽的时间之和，约束条件变为第一枚干扰弹相对导弹 M3 满足条件（4）（5）（6）（7）；需要注意的是，该步骤在计算对导弹 M1 的有效遮蔽时长时要去掉无人机 FY1 对 FY1 产生有效遮蔽的时间区间，确保结果的准确性。

然后，针对无人机 FY2 和 FY4 的同步优化问题，在使得导弹 M2 的有效遮蔽时长最长时给出烟幕弹的投放策略，需要对两架无人机的飞行速度、方向和第一枚烟幕弹的时间参数进行优化，可以使用问题四的多机单弹优化模型。

最后，确定无人机 FY2 和 FY4 剩余两枚烟幕弹的时间参数

综上，根据导弹与无人机的坐标位置关系与运动模型，设计上述先任务分配、再分组分层优化的优化策略，可以降低问题的维度，得到较优的目标值，最终计算得到有效遮蔽时长为 20.056319s，具体烟幕弹投放策略见 result3.xlsx。

十、模型的分析与检验

为了检验圆柱离散化思想的可行性，即问题一中将圆柱上下底面均分成若干个点来代表整个圆柱，进而判断是否被有效遮蔽的方法是否可行，本文计算了不同取点数时对应的问题一的有效遮蔽时长，进而对该方法的可行性进行分析。

如图所示为取点数从 2 点、8 点、16 点、32 点、50 点、100 点、200 点、400 点、600 点、800 点、1000 点所对的有效遮蔽时长：

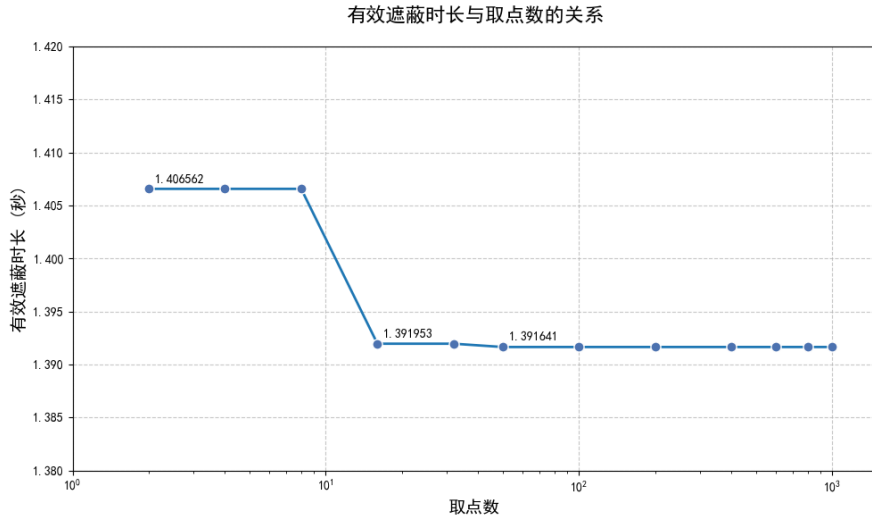


图 20 不同取点数对应的有效遮蔽时长

由图可知，取点数达到 50 点后直到 1000 点，有效遮蔽时长保持为 1.391641s 不变，因此，计算有效遮蔽时长时，**50 点为圆柱离散化最佳取点数**，可以在保证结果精确的同时尽可能减少计算量，提高模型的效率。

十一、模型的评价

11.1 模型的优点

- 1、模型在求解有效遮蔽时长时采用“**变步长搜索 + 二分法**”的方式计算遮蔽开始时间和结束时间，兼顾了搜索效率和结果精度；
- 2、对真目标圆柱作离散化处理，降低了模型复杂度；
- 3、充分挖掘产生有效遮蔽的隐含条件，极大地缩小解空间，提高优化模型的搜索效率

11.2 模型的缺点

该模型的计算复杂度随决策变量维度的增加而显著增长。

十二、参考文献

- [1] 罗瑞耀,王得霖,罗威,等.烟幕弹应对察打一体无人机的投放策略研究[J].光电技术应用,2022,37(06):90-98.
- [2] 王丹,吴孟达,毛紫阳.定日镜场的优化设计[J].数学建模及其应用,2024,13(01):30-43.DOI:10.19943/j.2095-3070.jmmia.2024.01.04.
- [3] DeepSeek, DeepSeek-V3, 深度求索 (DeepSeek), 2025-09-06

附录

附录 1

介绍：支撑材料的文件列表

- 1、AI 工具使用详.pdf
- 2、问题一所需函数 root_1.py
- 3、问题一求解代码 question1_1.py
- 4、问题二三四五所需函数 root_2.py
- 5、问题二求解代码 question2.py
- 6、问题三求解代码 question3.py
- 7、问题四求解代码 question4.py
- 8、结果文件 result1.xlsx、result2.xlsx、result3.xlsx

附录 2

介绍：问题三单机多弹投放策略优化模型

$$\begin{aligned}
 & \max T \\
 & \left\{ \begin{array}{l}
 0 \leq \theta \leq 2\pi \\
 70 \leq u \leq 140 \\
 t_{11}、t_{12}、t_{13} \geq 0 \\
 t_{21}、t_{22}、t_{23} \geq 0 \\
 t_{12} - t_{11} \geq 1 \\
 t_{13} - t_{12} \geq 1 \\
 0 \leq x_{P'} \leq x_{M_i'} \\
 k_1 x_{P'} \leq y_{P'} \leq k_2 x_{P'} + 200 \\
 z - \frac{1}{2} g t_2^2 > H_2 \\
 z - \frac{1}{2} g t_2^2 - 20v < H_1 \\
 X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i < x_A \\
 X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i > x_B \\
 \frac{\max\left(z - \frac{1}{2} g t_2^2 - 20v, H_2\right) - \left(z - \frac{1}{2} g t_2^2\right)}{v} \dots \\
 > \frac{\min\left(X_i - \frac{v_d(t_1 + t_2)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_A\right) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \\
 \frac{\min\left(z - \frac{1}{2} g t_2^2, H_1\right) - \left(z - \frac{1}{2} g t_2^2\right)}{v} \dots \\
 < \frac{\max\left(X_i - \frac{v_d(t_1 + t_2 + 20)}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} X_i, x_B\right) - X_i}{\frac{v_d X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}} - (t_1 + t_2) \\
 \frac{1}{2} g t_{21}^2 < z \\
 \frac{1}{2} g t_{22}^2 < z \\
 \frac{1}{2} g t_{23}^2 < z \\
 t_{11} + t_{21} < \frac{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}{v_d} \\
 t_{12} + t_{22} < \frac{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}{v_d} \\
 t_{13} + t_{23} < \frac{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}{v_d}
 \end{array} \right.
 \end{aligned}$$

附录 3: root_1.py

介绍: python 编写, 求解问题一所用到的类

```
import numpy as np
import math
import matplotlib.pyplot as plt
from matplotlib import font_manager
# 设置中文字体支持
plt.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans'] # 使用黑体, 如果系统中没有则使用DejaVu Sans
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
class SmokeObscurationModel:
    def __init__(self, num_lines=8):
        # 物理常数
        self.g = 9.8
        self.r_smoke = 10.0 # 云团有效半径
        self.v_sink = 3.0 # 云团下沉速度
        # 目标和导弹参数
        self.P_m1_0 = np.array([20000.0, 0.0, 2000.0]) # 导弹初始位置
        self.P_m2_0 = np.array([19000.0, 600.0, 2100.0])
        self.P_m3_0 = np.array([18000, -600, 1900])
        self.o = np.array([0.0, 0.0, 0.0]) # 假目标位置
        self.cylinder_center = np.array([0.0, 200.0, 0.0]) # 真目标圆柱中心
        self.cylinder_radius = 7.0 # 圆柱半径
        self.cylinder_height = 10.0 # 圆柱高度
        # 无人机参数
        self.FY1_0 = np.array([17800.0, 0.0, 1800.0]) # FY1 初始位置
        self.FY2_0 = np.array([12000.0, 1400.0, 1400.0])
        self.FY3_0 = np.array([6000.0, -3000.0, 700.0])
        self.FY4_0 = np.array([11000.0, 2000.0, 1800.0])
        self.FY5_0 = np.array([13000.0, -2000.0, 1300.0])
        self.v_drone = np.array([-120.0, 0.0, 0.0]) # 无人机速度
        # 烟幕弹参数
        self.t_drop = 1.5 # 投放时间
        self.tau_burst = 3.6 # 起爆延迟
        self.t_burst = self.t_drop + self.tau_burst # 起爆时间
        # 计算烟幕弹投放点和起爆点
        self.S0 = self.FY1_0 + self.v_drone * self.t_drop # 投放点
        x_b = self.S0[0] + self.v_drone[0] * self.tau_burst
        y_b = self.S0[1] + self.v_drone[1] * self.tau_burst
        z_b = self.S0[2] + self.v_drone[2] * self.tau_burst - 0.5 * self.g
        * self.tau_burst ** 2
        self.C0 = np.array([x_b, y_b, z_b]) # 云团中心起爆点
        # 计算导弹速度向量
        vec_to_0 = self.o - self.P_m1_0
```

```

dist_to_0 = np.linalg.norm(vec_to_0)
u_m = vec_to_0 / dist_to_0
self.v_m = 300 * u_m
# 创建圆柱母线
self.cylinder_lines = self.create_cylinder_lines(
    self.cylinder_center, self.cylinder_radius, self.cylinder_height, num_lines)
# 创建简化模型（上下底面圆心连线）
bottom_center = self.cylinder_center.copy()
top_center = self.cylinder_center + np.array([0, 0, self.cylinder_height])
self.simplified_model = (bottom_center, top_center)
# 创建质心模型
self.centroid_model = self.cylinder_center + np.array([0, 0, self.cylinder_height / 2])

def create_cylinder_lines(self, center, radius, height, num_lines=8):
    """创建圆柱的母线"""
    lines = []
    for i in range(num_lines):
        angle = 2 * np.pi * i / num_lines
        x = radius * np.cos(angle)
        y = radius * np.sin(angle)
        # 底部点
        bottom_point = np.array([center[0] + x, center[1] + y, center[2]])
        # 顶部点
        top_point = np.array([center[0] + x, center[1] + y, center[2] + height])
        lines.append((bottom_point, top_point))
    return lines

def cloud_center(self, t):
    """计算云团中心位置"""
    if t < self.t_burst:
        return None
    if t > self.t_burst + 20:
        return None
    dt = t - self.t_burst
    z = self.C0[2] - self.v_sink * dt
    return np.array([self.C0[0], self.C0[1], z])

def missile_position(self, t):
    """计算导弹位置"""
    return self.P_m1_0 + self.v_m * t

def angle_between(self, v1, v2):
    """计算两个向量之间的夹角"""

```

```

        cos_angle = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(
v2))

        cos_angle = np.clip(cos_angle, -1.0, 1.0)
        return np.arccos(cos_angle)
    def is_point_obscured(self, P, C, point):
        """检查一个点是否被遮蔽"""
        PC = C - P
        d = np.linalg.norm(PC)
        if d <= self.r_smoke:
            return True
        theta = np.arcsin(self.r_smoke / d)
        P_point = point - P
        alpha = self.angle_between(PC, P_point)
        return alpha <= theta
    def is_line_obscured(self, P, C, line):
        """检查一条线是否被遮蔽（两个端点都被遮蔽）"""
        bottom_point, top_point = line
        return (self.is_point_obscured(P, C, bottom_point) and
                self.is_point_obscured(P, C, top_point))
    def calculate_obscuration_time(self, check_function, t_start, t_end, i
nitial_step=0.01, min_step=0.0001,
                                max_iter=20):
        """计算遮蔽时间 - 使用变步长搜索方法"""
        # 第一步：使用较大步长找到遮蔽开始和结束的大致区间
        coarse_step = initial_step
        start_candidates = []
        end_candidates = []
        # 粗略扫描整个时间区间
        t_values = np.arange(t_start, t_end, coarse_step)
        obscured_values = [check_function(t) for t in t_values]
        # 找到所有状态变化点
        prev_obscured = obscured_values[0]
        for i, obscured in enumerate(obscured_values[1:], 1):
            if obscured != prev_obscured:
                # 状态变化点，记录时间
                change_time = t_values[i]
                if obscured: # 从不遮蔽变为遮蔽
                    start_candidates.append(change_time)
                else: # 从遮蔽变为不遮蔽
                    end_candidates.append(change_time)
                prev_obscured = obscured
        # 如果没有找到任何变化点
        if not start_candidates and not end_candidates:
            # 检查是否整个区间都被遮蔽或都不被遮蔽

```

```

        if obscured_values[0]:
            return t_end - t_start, t_start, t_end
        else:
            return 0.0, None, None
# 第二步: 对每个候选点使用二分法精确确定时间
def refine_boundary(t_low, t_high, is_start):
    """使用二分法精确确定边界时间"""
    for _ in range(max_iter):
        if t_high - t_low <= min_step:
            break
        t_mid = (t_low + t_high) / 2
        obscured_mid = check_function(t_mid)
        if is_start:
            # 寻找开始时间: 从 False 到 True 的变化点
            obscured_low = check_function(t_low)
            if obscured_low != obscured_mid:
                t_high = t_mid
            else:
                t_low = t_mid
        else:
            # 寻找结束时间: 从 True 到 False 的变化点
            obscured_high = check_function(t_high)
            if obscured_high != obscured_mid:
                t_low = t_mid
            else:
                t_high = t_mid
        return (t_low + t_high) / 2
# 精炼开始时间
refined_starts = []
for start_candidate in start_candidates:
    # 确定开始时间的搜索区间
    t_low = max(t_start, start_candidate - coarse_step)
    t_high = min(t_end, start_candidate + coarse_step)
    refined_start = refine_boundary(t_low, t_high, True)
    refined_starts.append(refined_start)
# 精炼结束时间
refined_ends = []
for end_candidate in end_candidates:
    # 确定结束时间的搜索区间
    t_low = max(t_start, end_candidate - coarse_step)
    t_high = min(t_end, end_candidate + coarse_step)
    refined_end = refine_boundary(t_low, t_high, False)
    refined_ends.append(refined_end)
# 处理找到的边界点

```

```

if refined_starts and refined_ends:
    # 取第一个开始时间和最后一个结束时间
    start_time = min(refined_starts)
    end_time = max(refined_ends)
    return end_time - start_time, start_time, end_time
elif refined_starts:
    # 只有开始时间, 没有结束时间 (遮蔽一直持续到最后)
    start_time = min(refined_starts)
    return t_end - start_time, start_time, t_end
elif refined_ends:
    # 只有结束时间, 没有开始时间 (遮蔽从最开始)
    end_time = max(refined_ends)
    return end_time - t_start, t_start, end_time
else:
    # 没有找到任何边界
    return 0.0, None, None
def check_cylinder_obscuration(self, t):
    """检查整个圆柱是否被遮蔽"""
    P = self.missile_position(t)
    C = self.cloud_center(t)
    if C is None:
        return False
    # 检查所有母线
    for line in self.cylinder_lines:
        if not self.is_line_obscurated(P, C, line):
            return False
    return True
def check_simplified_model_obscuration(self, t):
    """检查简化模型是否被遮蔽"""
    P = self.missile_position(t)
    C = self.cloud_center(t)
    if C is None:
        return False
    # 检查简化模型的两个点
    bottom_point, top_point = self.simplified_model
    return (self.is_point_obscurated(P, C, bottom_point) and
            self.is_point_obscurated(P, C, top_point))
def check_centroid_model_obscuration(self, t):
    """检查质心模型是否被遮蔽"""
    P = self.missile_position(t)
    C = self.cloud_center(t)
    if C is None:
        return False
    # 检查质心点

```

```

        return self.is_point_obscured(P, C, self.centroid_model)
def run_analysis(self):
    """运行遮蔽分析"""
    t_start = 8.00 # self.t_burst
    t_end = 9.50 # self.t_burst + 20.0
    # 计算圆柱遮蔽时间
    cylinder_duration, cylinder_start, cylinder_end = self.calculate_obscurat
    ion_time(
        self.check_cylinder_obscurat
    ion_time, t_start, t_end)
    # 计算简化模型遮蔽时间
    simplified_duration, simplified_start, simplified_end = self.calculate_obscurat
    ion_time(
        self.check_simplified_model_obscurat
    ion_time, t_start, t_end)
    # 计算质心模型遮蔽时间
    centroid_duration, centroid_start, centroid_end = self.calculate_obscurat
    ion_time(
        self.check_centroid_model_obscurat
    ion_time, t_start, t_end)
    # 计算每条母线的遮蔽时间
    line_durations = []
    line_starts = []
    line_ends = []
    for i, line in enumerate(self.cylinder_lines):
        def check_line(t, line=line):
            P = self.missile_position(t)
            C = self.cloud_center(t)
            if C is None:
                return False
            return self.is_line_obscured(P, C, line)
        duration, start, end = self.calculate_obscurat
    ion_time(check_line, t_start, t_end)
        line_durations.append(duration)
        line_starts.append(start)
        line_ends.append(end)
    # 找出最短的母线遮蔽时间
    min_line_duration = min(line_durations) if line_durations else 0
    min_line_index = line_durations.index(min_line_duration) if line_d
    urations else -1
    # 计算相对差值和比例
    if cylinder_duration > 0:
        simplified_abs_diff = abs(simplified_duration - cylinder_durat
    ion)
        simplified_rel_diff = simplified_abs_diff / cylinder_duration
    * 100
        centroid_abs_diff = abs(centroid_duration - cylinder_duration)

```

```

        centroid_rel_diff = centroid_abs_diff / cylinder_duration * 100
    else:
        simplified_abs_diff = abs(simplified_duration)
        simplified_rel_diff = 100 if simplified_duration > 0 else 0
        centroid_abs_diff = abs(centroid_duration)
        centroid_rel_diff = 100 if centroid_duration > 0 else 0
    # 打印结果
    print("=" * 50)
    print("烟雾遮蔽分析结果")
    print("=" * 50)
    print(f"\n 圆柱整体遮蔽时间: {cylinder_duration:.6f}s")
    if cylinder_start is not None:
        print(f"圆柱遮蔽开始时间: {cylinder_start:.6f}s")
        print(f"圆柱遮蔽结束时间: {cylinder_end:.6f}s")
    print(f"\n 简化模型(两点)遮蔽时间: {simplified_duration:.6f}s")
    if simplified_start is not None:
        print(f"简化模型开始时间: {simplified_start:.6f}s")
        print(f"简化模型结束时间: {simplified_end:.6f}s")
    print(f"\n 质心模型(一点)遮蔽时间: {centroid_duration:.6f}s")
    if centroid_start is not None:
        print(f"质心模型开始时间: {centroid_start:.6f}s")
        print(f"质心模型结束时间: {centroid_end:.6f}s")
    print(f"\n 最短母线遮蔽时间: {min_line_duration:.6f}s (母
线 {min_line_index + 1})")
    if min_line_index >= 0 and line_starts[min_line_index] is not None
:
        print(f"最短母线开始时间: {line_starts[min_line_index]:.6f}s")
        print(f"最短母线结束时间: {line_ends[min_line_index]:.6f}s")
    print(f"\n 简化模型与圆柱整体遮蔽时间差
异: {simplified_abs_diff:.6f}s")
    print(f"简化模型相对差异比例: {simplified_rel_diff:.2f}%")
    print(f"\n 质心模型与圆柱整体遮蔽时间差异: {centroid_abs_diff:.6f}s")
    print(f"质心模型相对差异比例: {centroid_rel_diff:.2f}%")
    # 判断简化模型是否可行
    if simplified_rel_diff < 5: # 差异小于5%认为可行
        print("\n 结论: 将真目标简化为上下底面圆心连线的方法是可行的")
    else:
        print("\n 结论: 将真目标简化为上下底面圆心连线的方法不可靠")
    # 判断质心模型是否可行
    if centroid_rel_diff < 5: # 差异小于5%认为可行
        print("结论: 将真目标简化为质心的方法是可行的")
    else:
        print("结论: 将真目标简化为质心的方法不可靠")

```

```

# 返回结果
    return cylinder_duration, simplified_duration, centroid_duration, min_line_duration, simplified_abs_diff, simplified_rel_diff, centroid_abs_diff, centroid_rel_diff

```

附录 4: question1_1.py

介绍: python 编写, 调用类求解问题一

```

from root_1_new import *
model = SmokeObscurationModel(num_lines=100)
model.run_analysis()

```

附录 5: root_2.py

介绍: python 编写, 求解问题二所需的类

```

import numpy as np
import math
import matplotlib.pyplot as plt
from matplotlib import rcParams
class Cartomizer:
    def __init__(self):
        self.g = 9.8
        self.o = np.array([0, 0, 0])
        self.r_cyl = 7
        self.h_cyl = 10
        self.cylinder_lines = self.create_cylinder_lines(
            np.array([0.0, 200.0, 0.0]), 7.0, 10.0)
        self.m_0 = np.array([[20000, 0, 2000],
                               [19000, 600, 2100],
                               [18000, -600, 1900]])

        # 计算导弹速度向量
        self.v_m1 = 300 * (self.o - self.m_0[0]) / np.linalg.norm(self.o - self.m_0[0])
        self.v_m2 = 300 * (self.o - self.m_0[1]) / np.linalg.norm(self.o - self.m_0[1])
        self.v_m3 = 300 * (self.o - self.m_0[2]) / np.linalg.norm(self.o - self.m_0[2])
        self.v_m = np.array([self.v_m1, self.v_m2, self.v_m3])
        self.fy_0 = np.array([[17800, 0, 1800],
                               [12000, 1400, 1400],
                               [6000, -3000, 700],

```

```

        [11000, 2000, 1800],
        [13000, -2000, 1300]))
self.simplified_model=(np.array([0,200,0]), np.array([0,200,10]))
self.v_sink = np.array([0, 0, -3])
self.v_fy1_q1 = np.array([-120, 0, 0])
self.burst_1_q1 = self.m_0[0] + self.v_fy1_q1 * (1.5 + 3.6) + self
.v_sink * 3.6
self.r_smoke = 10
self.tangent_plane_m1_up, self.tangent_plane_m1_bottom = self.miss
ile_cylinder_tangent_plane(self.m_0[0])
self.tangent_plane_m2_up, self.tangent_plane_m2_bottom = self.miss
ile_cylinder_tangent_plane(self.m_0[1])
self.tangent_plane_m3_up, self.tangent_plane_m3_bottom = self.miss
ile_cylinder_tangent_plane(self.m_0[2])
self.tangent_plane_m_up = np.array([self.tangent_plane_m1_up,self.
tangent_plane_m2_up,self.tangent_plane_m3_up])
self.tangent_plane_m_bottom = np.array([self.tangent_plane_m1_bott
om, self.tangent_plane_m2_bottom, self.tangent_plane_m3_bottom])
self.t1=np.zeros((1,15))
self.t2=np.zeros((1,15))
self.t_burst=self.t1+self.t2
self.theta_fy=np.zeros((1,5))
self.speed_fy=np.zeros((1,5))
self.v_fy=[];self.burst=[]
for k in range(5):
    v_fyk=np.array([math.cos(self.theta_fy[0,k])*self.speed_fy[0,k
], math.sin(self.theta_fy[0,k])*self.speed_fy[0,k], 0])
    self.v_fy.append(v_fyk)
    for b in range(3):
        burst_k = self.fy_0[k] + v_fyk * (self.t1[0,3*k+b] + self.
t2[0,3*k+b]) + self.v_sink * self.t2[0,3*k+b]
        self.burst.append(burst_k)
self.v_fy=np.asarray(self.v_fy)
self.burst = np.asarray(self.burst)
def create_cylinder_lines(self, center, radius, height, num_lines=8):
    """创建圆柱的母线"""
    lines = []
    for i in range(num_lines):
        angle = 2 * np.pi * i / num_lines
        x = radius * np.cos(angle)
        y = radius * np.sin(angle)
        # 底部点
        bottom_point = np.array([center[0] + x, center[1] + y, center[
2]])

```

```

        # 顶部点
        top_point = np.array([center[0] + x, center[1] + y, center[2]
+ height])
        lines.append((bottom_point, top_point))
    return lines
def missile_cylinder_tangent_plane(self, mk_0):
    """
    计算过导弹 k 轨迹的真目标圆柱体切面
    参数: mk_0 导弹初始位置, 格式为 np.array([x, y, z])
    返回: np.array([[a1, b1, c1, d1],[a, b, c, d]])
    上切面方程 a1x + b1y + c1z + d1 = 0 的系数 + 下切面方
程 ax + by + cz + d = 0 的系数
    """
    # z=0
    tangent_point_bottom_1, tangent_point_bottom_2 = self.find_tangent
_points(0, 0)
    tangent_point_bottom_1 = np.array([tangent_point_bottom_1[0], tang
ent_point_bottom_1[1], 0])
    tangent_point_bottom_2 = np.array([tangent_point_bottom_2[0], tang
ent_point_bottom_2[1], 0])
    tangent_plane_bottom_1 = self.calculate_plane_equation(self.o, mk_
0, tangent_point_bottom_1)
    tangent_plane_bottom_2 = self.calculate_plane_equation(self.o, mk_
0, tangent_point_bottom_2)
    z_bottom_1 = -
tangent_plane_bottom_1[0] * 0 - tangent_plane_bottom_1[1] * 200 - tangent_
plane_bottom_1[3]
    z_bottom_2 = -
tangent_plane_bottom_2[0] * 0 - tangent_plane_bottom_2[1] * 200 - tangent_
plane_bottom_2[3]
    if z_bottom_1 < z_bottom_2:
        tangent_plane_bottom = tangent_plane_bottom_1
        bottom_tangent_point = tangent_point_bottom_1
    else:
        tangent_plane_bottom = tangent_plane_bottom_2
        bottom_tangent_point = tangent_point_bottom_2
    # z=10
    tangent_point_top_1, tangent_point_top_2 = self.find_tangent_point
s(mk_0[0] * 10 / mk_0[2],
mk_0[1] * 10 / mk_0[2])
    tangent_point_top_1 = np.array([tangent_point_top_1[0], tangent_po
int_top_1[1], 10])

```

```

        tangent_point_top_2 = np.array([tangent_point_top_2[0], tangent_po
int_top_2[1], 10])
        tangent_plane_top_1 = self.calculate_plane_equation(self.o, mk_0,
tangent_point_top_1)
        tangent_plane_top_2 = self.calculate_plane_equation(self.o, mk_0,
tangent_point_top_2)
        z_top_1 = -
tangent_plane_top_1[0] * 0 - tangent_plane_top_1[1] * 200 - tangent_plane_
top_1[3]
        z_top_2 = -
tangent_plane_top_2[0] * 0 - tangent_plane_top_2[1] * 200 - tangent_plane_
top_2[3]
        if z_top_1 > z_top_2:
            tangent_plane_top = tangent_plane_top_1
            top_tangent_point = tangent_point_top_1
        else:
            tangent_plane_top = tangent_plane_top_2
            top_tangent_point = tangent_point_top_2
        # 存储切点信息用于可视化
        self.bottom_tangent_point = bottom_tangent_point
        self.top_tangent_point = top_tangent_point
        return (tangent_plane_top, tangent_plane_bottom)
def calculate_plane_equation(self, p1, p2, p3):
    """
    计算三个点确定的平面方程
    参数: p1, p2, p3: 三个点的坐标, 格式为 np.array([x, y, z])
    返回: np.array([a, b, c, d]) 平面方程  $ax + by + cz + d = 0$  的系数
    """
    v1 = p2 - p1
    v2 = p3 - p1
    normal = np.cross(v1, v2) # 计算法向量 (叉积)
    normal = normal / np.linalg.norm(normal) # 归一化法向量
    d = -np.dot(normal, p1) # 计算平面方程的常数项 d
    return np.array([normal[0], normal[1], normal[2], d]) # 返回平面方
程系数
    @staticmethod
    def find_tangent_points(cx1, cy1):
        """
        计算过圆外一点的圆的两个切点(x,y)
        参数: cx1, cy1
        返回: [t1,t2]
        """
        ox, oy, r = 0, 200, 7 # 圆心坐标和半径
        dx, dy = cx1 - ox, cy1 - oy

```

```

        d = math.sqrt(dx * dx + dy * dy) # 计算点到圆心的距离
        theta = math.atan2(dy, dx)
        alpha = math.acos(r / d) # 计算角度
        # 计算两个切点
        t1 = (ox + r * math.cos(theta + alpha), oy + r * math.sin(theta +
alpha))
        t2 = (ox + r * math.cos(theta - alpha), oy + r * math.sin(theta -
alpha))
        return [t1, t2]
    def t1_t2(self, mk_0, fyk_0, t_1, t_2, v_fy):
        """
        计算导弹 k 飞进烟雾圆柱遮挡区域的开始时刻 t1 与结束时刻 t2
        """
        # 爆炸点坐标
        burst = fyk_0 + v_fy * (t_1 + t_2) + np.array([0, 0, -
0.5 * self.g * t_2 ** 2])
        # print(burst)
        # 两圆柱公切面
        A1, B1, C1, D1, A2, B2, C2, D2=self.burst_truetarg_tangents_planes
(burst[0], burst[1])
        # print(A1, B1, C1, D1, A2, B2, C2, D2)
        # 导弹轨迹与两个公切面的交点
        point1, point2=self.find_intersections(mk_0, self.o, np.array([A1,
B1,C1,D1]), np.array([A2,B2,C2,D2]))
        # print(point1,point2)
        # 导弹到两个交点的时刻
        t1, t2 = self.calculate_time_to_points(mk_0, point1, point2)
        if t1 is None and t2 is not None:
            return (t_1+t_2,t2)
        elif t1 is not None and t2 is None:
            return (t_1+t_2,t1)
        elif t1 is None and t2 is None:
            return (0,0)
        else:
            return(min([t1,t2]),max([t1,t2]))
    def t3_t4(self, fyk_0, t_1, t_2, v_fy):
        """
        计算烟雾球与上下平面相切的时刻
        """
        # 爆炸点坐标
        burst = fyk_0 + v_fy * (t_1 + t_2) + np.array([0, 0, -
0.5 * self.g * t_2 ** 2])
        a3 = self.tangent_plane_m1_bottom[0]
        b3 = self.tangent_plane_m1_bottom[1]

```

```

        c3 = self.tangent_plane_m1_bottom[2]
        d3 = self.tangent_plane_m1_bottom[3]
        t3 = t_1 + t_2 + (a3 * burst[0] + b3 * burst[1] + c3 * burst[2] +
d3 - self.r_smoke * math.sqrt(
            a3 ** 2 + b3 ** 2 + c3 ** 2)) / 3 / c3
        a4 = self.tangent_plane_m1_up[0]
        b4 = self.tangent_plane_m1_up[1]
        c4 = self.tangent_plane_m1_up[2]
        d4 = self.tangent_plane_m1_up[3]
        t4 = t_1 + t_2 + (a4 * burst[0] + b4 * burst[1] + c4 * burst[2] +
d4 + self.r_smoke * math.sqrt(
            a4 ** 2 + b4 ** 2 + c4 ** 2)) / 3 / c4
    return t3,t4
def burst_truetarg_tangents_planes(self, xb, yb):
    """
    求起爆点在(xb,yb)时形成的圆柱与真目标圆柱的两个共切面解析方程系数
    """
    d = math.sqrt(xb ** 2 + (yb - 200) ** 2)
    u1 = xb / d
    u2 = (yb - 200) / d
    cos_alpha = 3 / d
    sin_alpha = math.sqrt(1 - (9 / (d ** 2)))
    A1 = u1 * cos_alpha - u2 * sin_alpha
    B1 = u2 * cos_alpha + u1 * sin_alpha
    D1 = 7 - 200 * B1
    A2 = u1 * cos_alpha + u2 * sin_alpha
    B2 = u2 * cos_alpha - u1 * sin_alpha
    D2 = 7 - 200 * B2
    return A1, B1, 0, D1, A2, B2, 0, D2
def find_intersections(self, line_point1, line_point2, plane1_coeffs,
plane2_coeffs):
    """
    求直线与两个平面的交点
    参数:
    line_point1: NumPy 数组 [x1, y1, z1], 直线上的第一个点
    line_point2: NumPy 数组 [x2, y2, z2], 直线上的第二个点
    plane1_coeffs: NumPy 数组 [A1, B1, C1, D1], 平面 1 的方程系数
    plane2_coeffs: NumPy 数组 [A2, B2, C2, D2], 平面 2 的方程系数
    返回:
    两个交点: 元组 (point1, point2), 其中 point1 是直线与平面 1 的交点,
point2 是直线与平面 2 的交点
    """
    # 确保输入为 NumPy 数组

```

```

        line_point1 = np.asarray(line_point1);line_point2 = np.asarray(line_point2)
        plane1_coeffs = np.asarray(plane1_coeffs);plane2_coeffs = np.asarray(plane2_coeffs)
        # 计算直线的方向向量
        direction = line_point2 - line_point1
        # 解与平面1的交点
        A1, B1, C1, D1 = plane1_coeffs
        normal1 = np.array([A1, B1, C1])
        # 检查直线是否与平面平行
        if np.abs(np.dot(normal1, direction)) < 1e-10:
            raise ValueError("直线与平面1平行, 无唯一交点")
        # 计算参数t - 修正符号问题
        t1 = -
(np.dot(normal1, line_point1) + D1) / np.dot(normal1, direction)
        # 计算交点坐标
        point1 = line_point1 + t1 * direction
        # 解与平面2的交点
        A2, B2, C2, D2 = plane2_coeffs
        normal2 = np.array([A2, B2, C2])
        # 检查直线是否与平面平行
        if np.abs(np.dot(normal2, direction)) < 1e-10:
            raise ValueError("直线与平面2平行, 无唯一交点")
        # 计算参数t - 修正符号问题
        t2 = -
(np.dot(normal2, line_point1) + D2) / np.dot(normal2, direction)
        # 计算交点坐标
        point2 = line_point1 + t2 * direction
        return point1, point2
def calculate_time_to_points(self,P0, P1, P2):
    """
    计算物体从初始点 P0 朝原点匀速运动到达点 P1 和 P2 的时刻。
    参数:
    P0: NumPy 数组 [x0, y0, z0], 初始点坐标
    P1: NumPy 数组 [x1, y1, z1], 第一个目标点坐标
    P2: NumPy 数组 [x2, y2, z2], 第二个目标点坐标
    返回:
    t1: 到达第一个点的时间 (秒)
    t2: 到达第二个点的时间 (秒)
    """
    # 定义速度
    speed = 300.0
    # 确保输入为NumPy 数组
    P0 = np.asarray(P0);P1 = np.asarray(P1);P2 = np.asarray(P2)

```

```

# 计算初始点到原点的距离
d0 = np.linalg.norm(P0)
if d0 == 0:
    raise ValueError("初始点位于原点，运动方向未定义")
# 计算标量 s 的函数
def compute_s(P0, P):
    return np.dot(P0, P) / (d0 ** 2)
# 检查点是否在直线上
def is_on_line(P0, P, s, tol=1e-6):
    expected = s * P0
    return np.allclose(P, expected, atol=tol)
# 计算到达点的时间
def compute_time(s):
    if s > 1:
        # raise ValueError("点位于运动方向的反方向，无法到达")
        return None
    return (1 - s) * d0 / speed
# 处理第一个点
s1 = compute_s(P0, P1)
if not is_on_line(P0, P1, s1):
    raise ValueError("点 1 不在运动路径上")
t1 = compute_time(s1)
# 处理第二个点
s2 = compute_s(P0, P2)
if not is_on_line(P0, P2, s2):
    raise ValueError("点 2 不在运动路径上")
t2 = compute_time(s2)
return t1, t2
def time_interval_intersection(self, t1, t2, t3, t4):
    """
    计算两个时间区间的交集
    t1, t2: 第一个时间区间的起点和终点
    t3, t4: 第二个时间区间的起点和终点
    返回:
    如果存在交集，返回交集的起点和终点 (start, end)
    如果没有交集，返回 None
    """
    # 确保时间区间是有效的 (t1 <= t2, t3 <= t4)
    if t1 > t2:
        t1, t2 = t2, t1
    if t3 > t4:
        t3, t4 = t4, t3
    # 计算交集的起点和终点
    start = max(t1, t3)

```

```

        end = min(t2, t4)
        # 检查是否存在交集
        if start <= end:
            return (start, end)
        else:
            return None
    def get_cylinder_meshgrid(self, theta_resolution=100, z_resolution=10)
:
        """获取圆柱体的网格坐标"""
        center = np.array([0, 200, 0])
        radius = self.r_cyl
        height = self.h_cyl
        theta = np.linspace(0, 2 * np.pi, theta_resolution)
        z_cyl = np.linspace(0, height, z_resolution)
        theta_grid, z_grid = np.meshgrid(theta, z_cyl)
        x_cyl = center[0] + radius * np.cos(theta_grid)
        y_cyl = center[1] + radius * np.sin(theta_grid)
        return x_cyl, y_cyl, z_grid
    def get_missile_trajectory_points(self, missile_start):
        """获取导弹轨迹上的点"""
        t = np.linspace(0, 1, 100)
        points = []
        for t_i in t:
            point = missile_start + t_i * (self.o - missile_start)
            points.append(point)
        return np.array(points)
    @staticmethod
    def is_point_in_triangle(A, B, C, P):
        # 提取A, B, C, P 的x 和y 坐标, 忽略z 坐标
        x1, y1 = A[0], A[1]
        x2, y2 = B[0], B[1]
        x3, y3 = C[0], C[1]
        x, y = P[0], P[1]
        # 计算向量
        v0 = (x3 - x1, y3 - y1)
        v1 = (x2 - x1, y2 - y1)
        v2 = (x - x1, y - y1)
        # 计算点积
        dot00 = v0[0] * v0[0] + v0[1] * v0[1]
        dot01 = v0[0] * v1[0] + v0[1] * v1[1]
        dot02 = v0[0] * v2[0] + v0[1] * v2[1]
        dot11 = v1[0] * v1[0] + v1[1] * v1[1]
        dot12 = v1[0] * v2[0] + v1[1] * v2[1]
        # 计算分母

```

```

denom = dot00 * dot11 - dot01 * dot01
if abs(denom) < 1e-10:
    # 三角形退化, 但已知三点不共线, 所以这里理论上不会发生
    return False
# 计算重心坐标 u 和 v
u = (dot11 * dot02 - dot01 * dot12) / denom
v = (dot00 * dot12 - dot01 * dot02) / denom
# 检查点是否在三角形内
return u >= 0 and v >= 0 and u + v <= 1
def calculate_obscuration_time(self, k_m, k_fy, n,
                                check_function, t_start, t_end, initial
_step=0.01,
                                min_step=0.0001, max_iter=10):
    """计算遮蔽时间 - 使用变步长搜索方法"""
    # 第一步: 使用较大步长找到遮蔽开始和结束的大致区间
    coarse_step = initial_step
    start_candidates = []
    end_candidates = []
    # 粗略扫描整个时间区间
    t_values = np.arange(t_start, t_end, coarse_step)
    obscured_values = [check_function(k_m, k_fy, n, t) for t in t_valu
es]

    # 找到所有状态变化点
    prev_obscured = obscured_values[0]
    for i, obscured in enumerate(obscured_values[1:], 1):
        if obscured != prev_obscured:
            # 状态变化点, 记录时间
            change_time = t_values[i]
            if obscured: # 从不遮蔽变为遮蔽
                start_candidates.append(change_time)
            else: # 从遮蔽变为不遮蔽
                end_candidates.append(change_time)
            prev_obscured = obscured
    # 如果没有找到任何变化点
    if not start_candidates and not end_candidates:
        # 检查是否整个区间都被遮蔽或都不被遮蔽
        if obscured_values[0]:
            return t_end - t_start, t_start, t_end
        else:
            return 0.0, None, None
    # 第二步: 对每个候选点使用二分法精确确定时间
    def refine_boundary(t_low, t_high, is_start):
        """使用二分法精确确定边界时间"""
        for _ in range(max_iter):

```

```

        if t_high - t_low <= min_step:
            break
        t_mid = (t_low + t_high) / 2
        obscured_mid = check_function(k_m, k_fy, n, t_mid)
        if is_start:
            # 寻找开始时间: 从 False 到 True 的变化点
            obscured_low = check_function(k_m, k_fy, n, t_low)
            if obscured_low != obscured_mid:
                t_high = t_mid
            else:
                t_low = t_mid
        else:
            # 寻找结束时间: 从 True 到 False 的变化点
            obscured_high = check_function(k_m, k_fy, n, t_high)
            if obscured_high != obscured_mid:
                t_low = t_mid
            else:
                t_high = t_mid
        return (t_low + t_high) / 2
# 精炼开始时间
refined_starts = []
for start_candidate in start_candidates:
    # 确定开始时间的搜索区间
    t_low = max(t_start, start_candidate - coarse_step)
    t_high = min(t_end, start_candidate + coarse_step)
    refined_start = refine_boundary(t_low, t_high, True)
    refined_starts.append(refined_start)
# 精炼结束时间
refined_ends = []
for end_candidate in end_candidates:
    # 确定结束时间的搜索区间
    t_low = max(t_start, end_candidate - coarse_step)
    t_high = min(t_end, end_candidate + coarse_step)
    refined_end = refine_boundary(t_low, t_high, False)
    refined_ends.append(refined_end)
# 处理找到的边界点
if refined_starts and refined_ends:
    # 取第一个开始时间和最后一个结束时间
    start_time = min(refined_starts)
    end_time = max(refined_ends)
    return end_time - start_time, start_time, end_time
elif refined_starts:
    # 只有开始时间, 没有结束时间 (遮蔽一直持续到最后)
    start_time = min(refined_starts)

```

```

        return t_end - start_time, start_time, t_end
    elif refined_ends:
        # 只有结束时间，没有开始时间（遮蔽从最开始）
        end_time = max(refined_ends)
        return end_time - t_start, t_start, end_time
    else:
        # 没有找到任何边界
        return 0.0, None, None
def check_cylinder_obscuriation(self, k_m, k_fy, n, t):
    """检查 k_m 导弹视角下，k_fy 无人机工作，每台无人机投掷 n 个烟雾弹，圆柱体
    是否被遮蔽"""
    P = self.missile_position(k_m, t)
    C = self.cloud_center(k_fy, n, t)
    if C is None:
        return False
    # 检查所有母线
    for line in self.cylinder_lines:
        if not self.is_line_obscured(P, C, line):
            return False
    return True
def check_simplified_model_obscuriation(self, k_m, k_fy, n, t):
    """检查 k_m 导弹视角下，k_fy 无人机工作，简化模型是否被遮蔽"""
    P = self.missile_position(k_m, t)
    C = self.cloud_center(k_fy, n, t)
    # print(P,C)
    if C is None:
        return False
    # 检查简化模型的两个点
    return self.is_line_obscured(P, C, self.simplified_model)
def missile_position(self, k, t):
    """计算导弹位置"""
    return self.m_0[k] + self.v_m[k] * t
def cloud_center(self, k, n, t):
    """计算云团中心位置"""
    if t < min(self.t_burst[0,k*3:(k*3+n)]):
        return None
    if t > max(self.t_burst[0,k*3:(k*3+n)]) + 20:
        return None
    center=[]
    for nk in range(n):
        dt = t - self.t_burst[0,k*3+nk]
        center_k=self.burst[k*3+nk] + self.v_sink * dt
        center.append(center_k)
    return center

```

```

def is_point_obscured(self, P, C, point):
    """检查一个点是否被遮蔽"""
    PC = C - P
    d = np.linalg.norm(PC)
    if d <= self.r_smoke:
        return True
    theta = np.arcsin(self.r_smoke / d)
    P_point = point - P
    alpha = self.angle_between(PC, P_point)
    # print(alpha <= theta)
    return alpha <= theta
def is_line_obscured(self, P, C, line):
    """检查一条线是否被遮蔽（两个端点都被遮蔽）"""
    bottom_point, top_point = line
    n = len(C) if hasattr(C, '__len__') and not isinstance(C, np.ndarray)
ay) else 1
    if n == 1:
        # 如果C是单个点，直接使用
        if hasattr(C, '__len__') and not isinstance(C, np.ndarray):
            C_point = C[0] # 从列表中取出点
        else:
            C_point = C
        return (self.is_point_obscured(P, C_point, bottom_point) and
                self.is_point_obscured(P, C_point, top_point))
    else:
        for k in range(n):
            if (self.is_point_obscured(P, C[k], bottom_point) and
                self.is_point_obscured(P, C[k], top_point)):
                return True
        return False
def angle_between(self, v1, v2):
    """计算两个向量之间的夹角"""
    cos_angle = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(
v2))
    cos_angle = np.clip(cos_angle, -1.0, 1.0)
    return np.arccos(cos_angle)
class Visualization:
    """可视化类"""
    def __init__(self):
        # 设置中文字体为宋体
        rcParams['font.sans-serif'] = ['SimSun'] # 使用宋体
        rcParams['axes.unicode_minus'] = False # 正常显示负号
    def visualize_cylinder_and_missile(self, cartomizer, missile_index=1):
        """可视化圆柱体和导弹轨迹"""

```

```

fig = plt.figure(figsize=(14, 10))
ax = fig.add_subplot(111, projection='3d')
# 获取导弹初始位置
missile_start = cartomizer.m_0[missile_index-1]
tangent_plane_bottom = cartomizer.tangent_plane_m_bottom[missile_index-1]
tangent_plane_top = cartomizer.tangent_plane_m_up[missile_index-1]
# 绘制圆柱
x_cyl, y_cyl, z_grid = cartomizer.get_cylinder_meshgrid()
ax.plot_surface(x_cyl, y_cyl, z_grid, alpha=0.3, color='gray', label='圆柱')
# 绘制导弹轨迹
missile_points = cartomizer.get_missile_trajectory_points(missile_start)
ax.plot(missile_points[:, 0], missile_points[:, 1], missile_points[:, 2],
        'g-', linewidth=3, label='导弹轨迹')
# 绘制切点
ax.scatter(*cartomizer.bottom_tangent_point, color='red', s=100, marker='x', label='底面切点')
ax.scatter(*cartomizer.top_tangent_point, color='blue', s=100, marker='x', label='顶面切点')
# 绘制相切平面
x_range = np.linspace(-20, 20, 20)
y_range = np.linspace(180, 220, 20)
x_grid, y_grid = np.meshgrid(x_range, y_range)
# 底面平面
a, b, c, d = tangent_plane_bottom
z_plane_bottom = (-a * x_grid - b * y_grid - d) / c
ax.plot_surface(x_grid, y_grid, z_plane_bottom, alpha=0.5, color='red', label='底面平面')
# 顶面平面
a, b, c, d = tangent_plane_top
z_plane_top = (-a * x_grid - b * y_grid - d) / c
ax.plot_surface(x_grid, y_grid, z_plane_top, alpha=0.5, color='blue', label='顶面平面')
# 设置坐标轴
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title(f'导弹 {missile_index} 的轨迹和相切平面')
ax.set_xlim(-20, 20)
ax.set_ylim(180, 220)
ax.set_zlim(0, 10)

```

```

# 添加图例
ax.legend()
plt.tight_layout()
# 保存图片
plt.savefig(f'导弹{missile_index}_相切平面.png', dpi=300, bbox_inches='tight')
print(f"图片已保存为 '导弹{missile_index}_相切平面.png'")
plt.show()
def visualize_tangent_planes(self, cartomizer, xb, yb):
    """
    可视化起爆点在(xb,yb)时形成的圆柱与真目标圆柱的两个公切面
    参数:
    cartomizer: Cartomizer 类的实例
    xb, yb: 起爆点的 x 和 y 坐标
    """

    fig = plt.figure(figsize=(14, 10))
    ax = fig.add_subplot(111, projection='3d')
    # 获取真目标圆柱的网格
    x_cyl, y_cyl, z_grid = self.get_cylinder_meshgrid(0, 200, cartomizer.r_cyl, cartomizer.h_cyl)
    ax.plot_surface(x_cyl, y_cyl, z_grid, alpha=0.3, color='blue', label='真目标圆柱')
    # 获取烟雾圆柱的网格
    x_smoke, y_smoke, z_smoke = self.get_cylinder_meshgrid(xb, yb, cartomizer.r_smoke, cartomizer.h_cyl)
    ax.plot_surface(x_smoke, y_smoke, z_smoke, alpha=0.3, color='red', label='烟雾圆柱')
    # 计算公切面
    A1, B1, C1, D1, A2, B2, C2, D2 = cartomizer.burst_truetarg_tangent_s_planes(xb, yb)
    # 绘制第一个公切面
    x_range = np.linspace(min(0, xb) - 20, max(0, xb) + 20, 20)
    y_range = np.linspace(min(200, yb) - 20, max(200, yb) + 20, 20)
    x_grid, y_grid = np.meshgrid(x_range, y_range)
    # 第一个公切面
    z_plane1 = (-
A1 * x_grid - B1 * y_grid - D1) / C1 if C1 != 0 else np.zeros_like(x_grid)
    ax.plot_surface(x_grid, y_grid, z_plane1, alpha=0.5, color='green', label='公切面 1')
    # 第二个公切面
    z_plane2 = (-
A2 * x_grid - B2 * y_grid - D2) / C2 if C2 != 0 else np.zeros_like(x_grid)
    ax.plot_surface(x_grid, y_grid, z_plane2, alpha=0.5, color='orange', label='公切面 2')

```

```

# 设置坐标轴
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title(f'起爆点({xb},{yb})的烟雾圆柱与真目标圆柱的公切面')
# 添加图例
ax.legend()
plt.tight_layout()
# 保存图片
plt.savefig(f'起爆点_{xb}_{yb}_公切面.png', dpi=300, bbox_inches='tight')
print(f"图片已保存为 '起爆点_{xb}_{yb}_公切面.png'")
plt.show()
def get_cylinder_meshgrid(self, center_x, center_y, radius, height):
    """
    生成圆柱体的网格点
    参数:
    center_x, center_y: 圆柱底面中心的 x 和 y 坐标
    radius: 圆柱半径
    height: 圆柱高度
    返回:
    x, y, z: 圆柱表面的网格点坐标
    """
    theta = np.linspace(0, 2 * np.pi, 100)
    z = np.linspace(0, height, 50)
    theta_grid, z_grid = np.meshgrid(theta, z)
    x = center_x + radius * np.cos(theta_grid)
    y = center_y + radius * np.sin(theta_grid)
    return x, y, z_grid

```

附录 5: question2.py

介绍: python 编写, 调用类求解问题二

```

from src.root_2_new import *
import numpy as np
import math
from concurrent.futures import ProcessPoolExecutor, as_completed
import multiprocessing
from itertools import product
import time
# 记录程序开始时间

```

```

program_start_time = time.time()
# 初始化模型
model2 = Cartomizer()
# 减少采样点数量
theta_fy1 = np.linspace(0, 0.5, 100)
speed_fy1 = np.linspace(70, 140, 100)
t1_fy1 = np.linspace(0, 20, 50)
t2_fy1 = np.linspace(0, 20, 50)
# 用于存储最大值的变量
max_during_time = 0
max_params = None
def process_batch(batch_args):
    """处理一批参数，减少函数调用开销"""
    batch_max_during_time = 0
    batch_max_params = None
    # 为每个进程创建独立的模型实例
    local_model = Cartomizer()
    for args in batch_args:
        theta_k, speed_k, t1_k, t2_k = args
        # 设置模型参数
        local_model.theta_fy[0] = theta_k
        local_model.speed_fy[0] = speed_k
        local_model.v_fy[0] = np.array([speed_k * math.cos(theta_k), speed_k * math.sin(theta_k), 0])
        local_model.t1[0] = t1_k
        local_model.t2[0] = t2_k
        local_model.burst[0] = local_model.fy_0[0] + local_model.v_fy[0] * (t1_k + t2_k) + local_model.v_sink * t2_k
        # 提前终止条件检查
        if not local_model.is_point_in_triangle(local_model.o, local_model.m_0[0], np.array([0, 200, 10]), local_model.burst[0]):
            continue
        try:
            t3_k, t4_k = local_model.t3_t4(local_model.fy_0[0], t1_k, t2_k, local_model.v_fy[0])
        except:
            continue
        h3_k = local_model.burst[0, 2] - 3 * t3_k
        h4_k = local_model.burst[0, 2] - 3 * t4_k
        if not (max([h4_k, local_model.burst[0, 2] - 20 * 3]) < min([h3_k, local_model.burst[0, 2]])):
            continue
        try:

```

```

        t1_new, t2_new = local_model.t1_t2(local_model.m_0[0], local_model.fy_0[0], t1_k, t2_k, local_model.v_fy[0])
    except:
        continue
    if t1_new == 0 or t2_new == 0:
        continue
    if not (max([t1_new, t3_k]) < min([t2_new, t4_k])):
        continue
    try:
        during_time_k, start_time_k, end_time_k = local_model.calculate_obscuraton_time(
            0, 0, 1, local_model.check_cylinder_obscuraton,
            max([t1_new, t3_k]), min([t2_new, t4_k]),
            initial_step=0.01, min_step=0.0001, max_iter=20
        )
    except:
        continue
    if during_time_k > batch_max_during_time:
        batch_max_during_time = during_time_k
        batch_max_params = (theta_k, speed_k, t1_k, t2_k)
    return batch_max_during_time, batch_max_params
def create_batches(param_combinations, batch_size=1000):
    """将参数组合分成批次"""
    for i in range(0, len(param_combinations), batch_size):
        yield param_combinations[i:i + batch_size]
if __name__ == '__main__':
    multiprocessing.freeze_support()
    # 记录参数生成开始时间
    param_generation_start = time.time()
    # 生成所有参数组合
    param_combinations = list(product(theta_fy1, speed_fy1, t1_fy1, t2_fy1))
    # 记录参数生成完成时间
    param_generation_time = time.time()
    print(f"参数生成完成, 耗时: {param_generation_time - param_generation_start:.2f} 秒")
    # 创建批次
    batches = list(create_batches(param_combinations, batch_size=100000))
    print(f"总参数组合数: {len(param_combinations)}")
    print(f"批次数: {len(batches)}")
    print("开始计算...")
    # 记录计算开始时间
    calculation_start_time = time.time()

```

```

# 使用多进程并行计算批次
with ProcessPoolExecutor(max_workers=multiprocessing.cpu_count()) as executor:
    futures = {executor.submit(process_batch, batch): i for i, batch in enumerate(batches)}
    for future in as_completed(futures):
        batch_max_during_time, batch_max_params = future.result()
        if batch_max_during_time > max_during_time:
            max_during_time = batch_max_during_time
            max_params = batch_max_params
    # 显示进度
    completed = sum(1 for f in futures if f.done())
    progress = completed / len(batches) * 100
    elapsed_time = time.time() - calculation_start_time
    estimated_total_time = elapsed_time / (completed / len(batches))
    if completed > 0 else 0
    remaining_time = estimated_total_time - elapsed_time
    print(f"进度: {progress:.2f}% ({completed}/{len(batches)}), "
          f"已耗时: {elapsed_time:.2f}秒, "
          f"预计剩余时间: {remaining_time:.2f}秒")
    # 记录计算完成时间
    calculation_end_time = time.time()
    calculation_time = calculation_end_time - calculation_start_time
    print(f"计算完成, 耗时: {calculation_time:.2f} 秒")
    # 记录程序结束时间
    program_end_time = time.time()
    total_time = program_end_time - program_start_time
    print(f"程序总运行时间: {total_time:.2f} 秒")
    # 输出结果
    if max_params is not None:
        theta_max, speed_max, t1_max, t2_max = max_params
        print(f"最大 during_time: {max_during_time}")
        print(f"参数: theta={theta_max/math.pi*180}, speed={speed_max}, t1={t1_max}, t2={t2_max}")
    else:
        print("未找到有效的 during_time")

```